COURSE 953101 | SE & CS COMPUTER PROGRAMMING

คู่มือสรุปเจาะลึกเตรียมสอบ Final Lab Exam (Java)

ครอบคลุมแล็บ Organized_Labs ทุกไฟล์ (36 ข้อ) + ทบทวนพื้นฐาน + ข้อสอบเก่า

รวบรวมและวิเคราะห์โค้ดภาษา Java ครบทั้ง 6 แล็บ (Lab 06A, 07A, 10A, 10B, 11A, 11B) พร้อมอธิบายหลักการทำงานทีละบรรทัด, ตาราง Trace ตัวแปร, โจทย์พลิกแพลงแบบต่างๆ, และกับดักข้อผิดพลาดในห้องสอบ

วิชา:	953101 Computers and Programming (Java)
หลักสูตร:	Software Engineering (SE), College of Arts, Media and Technology
เนื้อหาที่ครอบคลุม:	While Loop, For Loop, Nested Loops, Value-Returning & Void Methods, 1D & 2D Arrays
จำนวนไฟล์โปรแกรม:	36 โปรแกรม (จาก Organized_Labs) + 3 Model Exam Solutions
จัดทำเพื่อ:	นักศึกษาเตรียมสอบ Final Lab Exam และ AI Agent เพื่อการตรวจสอบความถูกต้อง 100%

สารบัญเนื้อหา (Table of Contents)

บทที่ 0: ทบทวนพื้นฐาน Java ก่อนสอบ

printf Specifiers (%2f, %4d, %-5d), if-else, Operators (%, ^, &&), Scanner Buffer

บทที่ 1: While Loops (Lab 06A - 7 ข้อ)

Trace, CountDown, SumUntilZero (Sentinel), InchToCm, Divisibility (XOR), GuessNumber, Calculator

บทที่ 2: For Loops & Nested (Lab 07A - 7 ข้อ)

Square Loop, Even/Odd Sums, Two-Variable Loop, Break/Continue, GCD, Primes, Centered Pyramid

บทที่ 3: Methods Part 1 (Lab 10A - 5 ข้อ)

Max Method, SumRange, TempConverter, Euclidean Distance, Triangle Classification

บทที่ 4: Methods Part 2 (Lab 10B - 6 ข้อ)

Pass-by-Value Mechanics, Modular GCD, isPrime Helper, Perfect Numbers, TextBox, MultiTable

บทที่ 5: 1D Arrays (Lab 11A - 6 ข้อ)

Array Indexing, Dynamic Input, Single-pass Stats, Linear Search, Filtering, In-place Reversal

บทที่ 6: 2D Arrays (Lab 11B - 5 ข้อ)

Matrix Coordinates, Resource-safe Input, Total & Row Sums, Row Maximums, 2D Early Exit Search

บทที่ 7: เฉลยข้อสอบ Final Lab Exam

Model Answers จากข้อสอบจริง: 1. Temp Stats 2. Shipping Cost 3. Weekly Rainfall Report

บทที่ 8: Top 10 ข้อผิดพลาดร้ายแรงในห้องสอบ (Fatal Exam Pitfalls)

เจาะลึก 10 จุดตัดคะแนน: Integer Division, Bounds Exception, Sentinel Order, String ==, Loop Semicolons

บทที่ 0: ทบทวนพื้นฐาน Java สำหรับห้องสอบ (Basics & Syntax)

สรุปหัวใจสำคัญของพื้นฐานภาษา Java ที่โจทย์แล็บและข้อสอบนำมาใช้ผสมผสานกับ Loops, Methods และ Arrays

0.1 printf Format Specifiers (การจัดรูปแบบ Output)

System.out.printf() ใช้จัดรูปแบบข้อความ ตัวเลขทศนิยม และการจัดระยะตาราง ซึ่งออกสอบในแทบทุกแล็บ

Specifier	คำอธิบาย / ชนิดข้อมูล	ตัวอย่างโค้ด	ผลลัพธ์ที่ได้
%d	จำนวนเต็ม (byte, short, int, long)	System.out.printf("Count: %d", 42);	Count: 42
%f	ทศนิยม (float, double) ค่าเริ่มต้น 6 ตำแหน่ง	System.out.printf("Val: %f", 3.14);	Val: 3.140000
%.2f	ทศนิยมปัดเศษเหลือ 2 ตำแหน่ง (พบบ่อยสุดในข้อสอบ)	System.out.printf("Avg: %.2f", 3.14159);	Avg: 3.14
%.1f	ทศนิยมปัดเศษเหลือ 1 ตำแหน่ง	System.out.printf("Temp: %.1f", 37.89);	Temp: 37.9
%5d	จำนวนเต็ม จัดขวา กว้างอย่างน้อย 5 ช่อง (เติมช่องว่างซ้าย)	System.out.printf("[%5d]", 7);	[7]
%-5d	จำนวนเต็ม จัดซ้าย กว้างอย่างน้อย 5 ช่อง (เติมช่องว่างขวา)	System.out.printf("[%-5d]", 7);	[7]
%10.2f	ทศนิยมกว้างรวม 10 ช่อง (รวมจุด) ทศนิยม 2 ตำแหน่ง ชิดขวา	System.out.printf("[%10.2f]", 45.5);	[45.50]
%-15s	String จัดชิดซ้าย กว้างอย่างน้อย 15 ช่อง (ใช้ทำหัวตาราง)	System.out.printf("%-15s", "Name");	Name
%n หรือ \n	ขึ้นบรรทัดใหม่ (Platform-independent newline)	System.out.printf("Done%n");	Done (ขึ้นบรรทัดใหม่)

> 💡 **คำแนะนำสำคัญ**
>
> **กฎเหล็ก:** ใน printf เครื่องหมายเปอร์เซ็นต์ต้องตรงกับชนิดตัวแปร หากนำ double ไปใส่ใน %d จะเกิด `IllegalFormatConversionException` ทันที!

0.2 คำสั่งเงื่อนไข (if-else, Ternary Operator, switch-case)

การตัดสินใจแยกสาขาการทำงานตามเงื่อนไขทางตรรกะ

if - else if - else (การตัดสินใจแบบช่วงชั้น)

```
if (score >= 80) {
    grade = "A";
} else if (score >= 70) {
    grade = "B";
} else if (score >= 60) {
    grade = "C";
} else {
    grade = "F";
}
```

Ternary Operator (if-else แบบย่อในบรรทัดเดียว)

```
// รูปแบบ: (เงื่อนไข) ? ค่าถ้าจริง : ค่าถ้าเท็จ;
String status = (age >= 18) ? "Adult" : "Minor";
double cost = (isMember) ? price * 0.9 : price;
int max = (a > b) ? a : b;
```

switch-case (เลือกทำงานตามค่าคงที่)

```
switch (choice) {
    case 1:
        System.out.println("Add");
        break; // ห้ามลืม break!
    case 2:
        System.out.println("Subtract");
        break;
    default:
        System.out.println("Invalid choice");
        break;
}
```

> 💡 **คำแนะนำสำคัญ**
>
> **จุดระวัง:** ถ้าลืมใส่ `break;` ใน switch-case โปรแกรมจะไหลทะลุไปทำงานใน case ถัดไปเรื่อยๆ เรียกว่า *Fall-through bug*

0.3 ตัวดำเนินการที่ต้องระวังเป็นพิเศษ (Operators)

ตัวดำเนินการที่นักศึกษามักสับสนและผิดพลาดในข้อสอบ

ตัวดำเนินการ	ชื่อ / การทำงาน	ตัวอย่างที่ถูกต้อง	ข้อผิดพลาดที่มักพบ
/	การหาร (Integer Division)	5.0 / 2 = 2.5 (double) sum / n	5 / 2 = 2 (เศษถูกตัดทิ้งทั้งหมด!)
%	โมดูโล (เศษจากการหาร)	10 % 3 = 1 n % 2 == 0 (เช็คเลขคู่)	ลืมคิดกรณีเลขติดลบ เช่น -7 % 3 = -1
^	Bitwise XOR (ไม่ใช่ยกกำลัง!)	true ^ false = true true ^ true = false	เอา 2^3 มาคิดว่าจะได้ 8 ใน Java ได้ 1 (2 XOR 3)
Math.pow()	การยกกำลังใน Java	Math.pow(2, 3) = 8.0 (คืนค่าเป็น double)	ลืม cast เป็น int เมื่อต้องการจำนวนเต็ม: (int)Math.pow(2,3)
&&	Logical AND (จริงทั้งคู่)	score >= 0 && score <= 100	เขียนแบบคณิตศาสตร์: 0 <= score <= 100 (Java Syntax Error!)
||	Logical OR (จริงอย่างน้อยหนึ่ง)	ch == 'y' || ch == 'Y'	เขียน ch == 'y' || 'Y' (ผิดไวยากรณ์)
!	Logical NOT (กลับค่าความจริง)	!found (เท่ากับ found == false)	สับสนลำดับการทำงานเมื่อใช้ร่วมกับตัวอื่น
==	เปรียบเทียบค่า / Reference	num == 5 (สำหรับ primitive)	ใช้ str == "yes" เปรียบเทียบ String (ต้องใช้ .equals())

> 💡 **คำแนะนำสำคัญ**
>
> **สูตรสำคัญ:** คำนวณค่าเฉลี่ยห้ามเขียน `sum / count` เด็ดขาด ให้เขียน `(double) sum / count` หรือ `sum * 1.0 / count` เสมอ

0.4 การรับค่าจากคีย์บอร์ดด้วย Scanner

การจัดการ Scanner และการเคลียร์ Buffer

รูปแบบการใช้งาน Scanner มาตรฐาน

```
import java.util.Scanner;

Scanner sc = new Scanner(System.in);
int age = sc.nextInt();          // รับเลขจำนวนเต็ม
double price = sc.nextDouble();  // รับเลขทศนิยม
String word = sc.next();         // รับข้อความคำเดียว (ตัดที่ space)
String line = sc.nextLine();     // รับทั้งบรรทัด (รวม space)
```

กับดัก Scanner Buffer: nextInt() ตามด้วย nextLine()

```
// ถ้าต้องรับตัวเลขแล้วตามด้วยข้อความทั้งบรรทัด:
int id = sc.nextInt();
sc.nextLine(); // สำคัญมาก! เพื่อเคลียร์ Enter (\n) ที่ค้างใน Buffer ทิ้งไป
String fullName = sc.nextLine(); // จึงจะสามารถรับข้อความบรรทัดใหม่ได้จริง
```

> 💡 **คำแนะนำสำคัญ**
>
> **ข้อควรปฏิบัติในแล็บ:** สร้าง Scanner เพียงตัวเดียวใน `main()` หาก Method ย่อยต้องการอ่านค่า ให้ส่ง Scanner เป็น Parameter เข้าไป อย่าสร้าง `new Scanner(System.in)` ซ้ำๆ ภายใน Method ย่อย

บทที่ 1: While Loops (Lab 06A — 7 Exercises)

ภาพรวมเนื้อหา: การควบคุมลูปแบบเซ็คเงื่อนไขก่อนทำ, Sentinel-controlled loop, Counter loop, และ Menu-driven program

ข้อที่ 1: Ex 1: การ Trace การทำงานของ While Loop พื้นฐาน

Loop Flow & Counter `Lab06A_WhileLoop/Task1_Trace.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

ทำความเข้าใจวงจรการทำงาน (Execution Cycle) ของคำสั่ง while, การตรวจสอบเงื่อนไขก่อนเข้าลูป, และการเปลี่ยนแปลงค่าตัวแปรนับรอบ (Counter)

ซอร์สโค้ดฉบับสมบูรณ์ (Task1_Trace.java):

```
// SE101 - Lab 6A: Task 1 (Exercise 1)
// Trace the while Loop

public class Task1_Trace {
    public static void main(String[] args) {
        int count = 0;
        while (count < 4) {
            System.out.println("count = " + count);
            count++;
        }
        System.out.println("Done");
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. ประกาศตัวแปรนับรอบ `int count = 0;` เป็นค่าเริ่มต้น
2. เข้าสู่เงื่อนไข `while (count < 4)`: โปรแกรมจะตรวจสอบว่า count น้อยกว่า 4 หรือไม่ หากเป็นจริง (true) จึงจะเข้าไปทำงานในปีกกา
3. ภายในลูป: พิมพ์ข้อความ `"Count is " + count` ออกทางหน้าจอ
4. เพิ่มค่า `count++` ทีละ 1 (หากลืมบรรทัดนี้จะกลายเป็น Infinite Loop ทันที)
5. เมื่อ count กลายเป็น 4 การตรวจสอบเงื่อนไข `4 < 4` จะเป็นเท็จ (false) ทำให้โปรแกรมหลุดออกจากลูป

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

```
ตารางจำลองการทำงาน (Trace Table):
รอบที่ | ค่า count ก่อนตรวจ | เงื่อนไข (count < 4) | ผลการพิมพ์ทางจอภาพ | ค่า count หลังเพิ่ม (count++)
-------------------------------------------------------------------------------------
  1    |        0        |    0 < 4 (True)     | Count is 0       |            1
  2    |        1        |    1 < 4 (True)     | Count is 1       |            2
  3    |        2        |    2 < 4 (True)     | Count is 2       |            3
  4    |        3        |    3 < 4 (True)     | Count is 3       |            4
  -    |        4        |    4 < 4 (False)    | (หลุดออกจากลูป)   |            4

ผลลัพธ์หลังจบลูป: ตัวแปร count มีค่าสุดท้ายเท่ากับ 4 (วนทำงานจริง 4 รอบ)
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **เปลี่ยนเงื่อนไขเป็น <= :** ถ้าแก้เป็น `while (count <= 4)` ลูปจะวน 5 รอบ (count = 0, 1, 2, 3, 4) และค่าสุดท้ายของ count จะเป็น 5
- **เริ่มนับจาก 1:** `int count = 1; while (count <= 4)` จะได้รอบที่ count เป็น 1, 2, 3, 4 เช่นกัน แต่วิธีนี้เหมาะกับการนับจำนวนชิ้นงาน

- **เขียนแบบ Prefix Increment ในเงื่อนไข:** `while (++count < 4)` ค่า count จะถูกบวก 1 *ก่อน* นำไปตรวจสอบเงื่อนไข

⚠️ จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)

ข้อสอบ Trace ข้อเขียนมักถามว่า 'หลังจบลูป ตัวแปร count จะมีค่าเท่าใด?' นักศึกษามักตอบ 3 (เพราะเห็นตัวเลขสุดท้ายที่พิมพ์คือ 3) แต่คำตอบที่ถูกต้องคือ **4** เพราะต้องเพิ่มเป็น 4 ก่อนจึงจะทำให้เงื่อนไขเป็นเท็จและหลุดลูปได้

ข้อที่ 2: Ex 2: การนับถอยหลังและการตรวจสอบเลขคู่-คี่ (Countdown & Modulo)

Decrement Loop & If-Else Branching `Lab06A_WhileLoop/CountDown.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

ฝึกการสร้างลูปนับถอยหลัง (Decrement) และการนำตัวดำเนินการโมดูโล (%) มาใช้แยกแยะเลขคู่ (Even) และเลขคี่ (Odd)

ซอร์สโค้ดฉบับสมบูรณ์ (CountDown.java):

```
public class CountDown {
    public static void main(String[] args) {
        int i = 10;
        while (i >= 2) {
            System.out.println(i + (i % 2 == 0 ? " Even" : " Odd"));
            i--;
        }
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. กำหนดค่าเริ่มต้น `int i = 10;` (เริ่มจากค่ามาก)
2. เงื่อนไข `while (i >= 2)`: วนทำงานตราบใดที่ i ยังมีค่ามากกว่าหรือเท่ากับ 2
3. ตรวจสอบเลขคู่/คี่: ใช้ `if (i % 2 == 0)` ถ้าเศษเป็น 0 แปลว่าเป็นเลขคู่ ให้พิมพ์คำว่า 'Even' ถ้าไม่ใช่พิมพ์ 'Odd'
4. ลดค่าตัวแปร: `i--` เพื่อให้ค่านับถอยหลังลงไปเรื่อยๆ เข้าใกล้จุดสิ้นสุด
5. เมื่อ i ลดลงเหลือ 1 เงื่อนไข `1 >= 2` จะเป็นเท็จ ทำให้หยุดการทำงาน

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

ลำดับการประมวลผล:

```
i = 10 -> 10 % 2 == 0 (True)  -> พิมพ์: 10 Even  -> i ลดเหลือ 9
i = 9  -> 9 % 2 == 0 (False) -> พิมพ์: 9 Odd    -> i ลดเหลือ 8
i = 8  -> 8 % 2 == 0 (True)  -> พิมพ์: 8 Even  -> i ลดเหลือ 7
...
i = 2  -> 2 % 2 == 0 (True)  -> พิมพ์: 2 Even  -> i ลดเหลือ 1
i = 1  -> 1 >= 2 (False)     -> จบลูปทันที
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **เขียนแบบย่อด้วย Ternary:** `System.out.println(i + (i % 2 == 0 ? " Even" : " Odd"));`
- **ลดทีละ 2 เพื่อพิมพ์เฉพาะเลขคู่:** `for (int i = 10; i >= 2; i -= 2)` ไม่ต้องใช้ if เช็ค
- **นับถอยหลังถึง 0:** เปลี่ยนเงื่อนไขเป็น `while (i >= 0)` ตัวเลข 0 จะถือว่าเป็น Even

⚠️ จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)

ระวังการเขียนเครื่องหมายเปรียบเทียบกลับด้าน เช่น เผลอเขียน `i <= 2` ทำให้เงื่อนไขเป็นเท็จตั้งแต่รอบแรก (10 <= 2 คือ false) ส่งผลให้ลูปไม่ทำงานเลยแม้แต่รอบเดียว

ข้อที่ 3: Ex 3: Sentinel Loop สะสมผลรวมจนกว่าจะพบศูนย์

Sentinel-Controlled Loop & Statistics | `Lab06A_WhileLoop/SumUntilZero.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

เรียนรู้รูปแบบ Sentinel-Controlled Loop ซึ่งเป็น Pattern สำคัญอันดับหนึ่งของข้อสอบแล็บ โดยรับข้อมูลจากผู้ใช้ไปเรื่อยๆ จนกว่าจะเจอเลขระบุการหยุด (Sentinel Value)

ซอร์สโค้ดฉบับสมบูรณ์ (SumUntilZero.java):

```
// SE101 - Lab 6A: Exercise 3 (SumUntilZero.java)
import java.util.Scanner;

public class SumUntilZero {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int sum = 0;
        int count = 0;
        int num;

        while (true) {
            System.out.print("Enter a number (0 to stop): ");
            num = scanner.nextInt();

            if (num == 0) {
                break;
            }

            sum += num;
            count++;
        }

        System.out.println("You entered " + count + " numbers.");
        System.out.println("Sum = " + sum);
        scanner.close();
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. สร้าง Scanner และตัวแปรสะสมผลรวม `sum = 0`, ตัวแปรนับจำนวน `count = 0`
2. ใช้ลูป `while (true)` เพื่อรับค่าวนซ้ำอย่างไม่จำกัดรอบ
3. รับค่าจากผู้ใช้: `int num = sc.nextInt();`
4. **ตรวจสอบ Sentinel ทันที:** `if (num == 0) break;` ออกจากลูปทันทีถ้าผู้ใช้ป้อน 0
5. หากไม่ใช่ 0 ให้นำไปสะสม `sum += num;` และนับเพิ่ม `count++;`
6. หลังจบลูป: ป้องกันการหารด้วยศูนย์ `if (count > 0)` ก่อนคำนวณค่าเฉลี่ย `(double) sum / count`

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

ตัวอย่างการป้อนข้อมูลและผลลัพธ์:

```
Enter number: 8   -> sum = 8,  count = 1
Enter number: 12  -> sum = 20, count = 2
Enter number: 5   -> sum = 25, count = 3
Enter number: 0   -> พบเลข 0 สั่ง break ทันที!
----------------------------
```

```
Total sum   = 25
Total count = 3
Average     = 8.33 (คำนวณจาก 25.0 / 3)
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **เปลี่ยน Sentinel:** ข้อสอบอาจเปลี่ยนเป็น -1, -999 หรือ 9999
- **แบบไม่ใช้ break:** `int num = sc.nextInt(); while (num != 0) { sum += num; count++; num = sc.nextInt(); }` (รูปแบบ Read-before-loop)
- **หาค่า Min และ Max ไปด้วย:** ประกาศตัวแปร min, max แล้วอัปเดตทุกรอบที่ไม่ใช่ sentinel

⚠️ จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)

ข้อผิดพลาดร้ายแรง: เอาคำสั่ง `sum += num;` ไปไว้ *ก่อน* คำสั่งตรวจสอบ sentinel ส่งผลให้เลข sentinel (เช่น 0 หรือ -999) ถูกบวกเข้าไปใน sum และถูกนับใน count ด้วย ทำให้ผลการคำนวณทั้งหมดผิดพลาด

ข้อที่ 4: Ex 4: ตารางแปลงหน่วยนิ้วเป็นเซนติเมตร (Unit Conversion Table)

Formatted Table Output & Step Loop | `Lab06A_WhileLoop/InchToCm.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

สร้างตารางแสดงผลลัพธ์สองคอลัมน์ที่มีการจัดระยะช่องไฟอย่างเป็นระเบียบด้วยคำสั่ง `System.out.printf` และการก้าวข้ามรอบแบบสเต็ป (Step increment)

ซอร์สโค้ดฉบับสมบูรณ์ (InchToCm.java):

```
// SE101 - Lab 6A: Exercise 4 (InchToCm.java)
public class InchToCm {
    public static void main(String[] args) {
        System.out.printf("%-10s%-15s%n", "Inches", "Centimeters");
        int inches = 1;
        while (inches <= 199) {
            double cm = inches * 2.54;
            System.out.printf("%-10d%-15.2f%n", inches, cm);
            inches += 2;
        }
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. พิมพ์ส่วนหัวตาราง (Table Header) โดยกำหนดความกว้างคอลัมน์ให้ตรงกับข้อมูลที่จะพิมพ์
2. กำหนดค่าเริ่มต้น `inches = 1`
3. วนลูป `while (inches <= 199)` เพื่อแปลงค่าเฉพาะเลขคี่ตั้งแต่ 1 ถึง 199
4. คำนวณความยาวเป็นเซนติเมตรด้วยสูตร `cm = inches * 2.54;`
5. แสดงผลด้วย `System.out.printf("%-10d%-15.2f\n", inches, cm);`
6. เพิ่มค่าทีละ 2: `inches += 2;` (กระโดดจาก 1 -> 3 -> 5 -> ...)

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

```
ตัวอย่างหน้าจอ Output:
Inches    Centimetres
-------------------------
1         2.54
3         7.62
5         12.70
...
197       500.38
199       505.46

อธิบาย Format:
%-10d : จำนวนเต็ม กว้าง 10 ช่อง ชิดซ้าย
%-15.2f: ทศนิยม 2 ตำแหน่ง กว้าง 15 ช่อง ชิดซ้าย
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **ตารางแปลงคู่ขนานสองทาง:** แสดงฝั่งซ้าย Inches -> Cm และฝั่งขวา Cm -> Inches ไปพร้อมกันในบรรทัดเดียว
- **แปลงเป็น For Loop:** `for (int inches = 1; inches <= 199; inches += 2)`

- **รับขอบเขตจากผู้ใช้:** รับค่า minInches, maxInches และ stepSize จาก Scanner

⚠️ จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)

ลืมใส่เครื่องหมายลบใน Format Specifier เช่น เขียน `%10d` แทน `%-10d` ทำให้ข้อความจัดชิดขวาแทนที่จะชิดซ้าย ส่งผลให้หัวตารางเบี้ยวไม่ตรงกับข้อมูลตัวเลขด้านล่าง

ข้อที่ 5: Ex 5: ตัวเลขที่หารลงตัวด้วย 5 หรือ 7 แต่ไม่ใช่ทั้งสองตัว (XOR Operator)

Logic Operators & Line-break Formatting `Lab06A_WhileLoop/Divisibility.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

ฝึกใช้ตัวดำเนินการตรรกะระดับบิต Exclusive OR (^) กับการตัดขึ้นบรรทัดใหม่ทุกๆ 10 จำนวน

ซอร์สโค้ดฉบับสมบูรณ์ (Divisibility.java):

```
// SE101 - Lab 6A: Exercise 5 (Divisibility.java)
import java.util.Scanner;

public class Divisibility {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter n: ");
        int n = scanner.nextInt();

        int count = 0;
        int num = 1;

        while (num <= n) {
            if ((num % 5 == 0) ^ (num % 7 == 0)) {
                System.out.print(num + " ");
                count++;
                if (count % 10 == 0) {
                    System.out.println();
                }
            }
            num++;
        }

        if (count % 10 != 0) {
            System.out.println();
        }
        System.out.println("End.");
        scanner.close();
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. วนลูปตรวจสอบตัวเลขตั้งแต่ 100 ถึง 200 ด้วย `int num = 100; while (num <= 200)`
2. สร้างเงื่อนไขตรวจสอบ: `(num % 5 == 0) ^ (num % 7 == 0)`
   เครื่องหมาย ^ (XOR) จะเป็นจริงก็ต่อเมื่อเงื่อนไขฝั่งใดฝั่งหนึ่งเป็นจริงเพียงฝั่งเดียวเท่านั้น
3. หากเข้าเงื่อนไข: พิมพ์ตัวเลขตามด้วยช่องว่าง `System.out.print(num + " ");` และเพิ่มตัวนับ `count++;`
4. ขึ้นบรรทัดใหม่ทุก 10 ตัว: ตรวจสอบ `if (count % 10 == 0) System.out.println();`
5. เพิ่มค่า `num++;` ไปตรวจตัวเลขถัดไป

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

```
การทดสอบค่าความจริงของเงื่อนไข XOR:
- ตัวเลข 100: 100 % 5 == 0 (True), 100 % 7 == 0 (False) -> True ^ False = True  -> พิมพ์!
- ตัวเลข 105: 105 % 5 == 0 (True), 105 % 7 == 0 (True)  -> True ^ True  = False -> ไม่พิมพ์! (เพราะหารลงทั้งคู่)
```

```
- ตัวเลข 101: 101 % 5 == 0 (False), 101 % 7 == 0 (False)-> False ^ False = False-> ไม่พิมพ์!
- ตัวเลข 140: หารลงทั้ง 5 และ 7 (35 * 4) -> True ^ True = False -> ไม่พิมพ์!
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **เขียนโดยไม่ใช้ ^ :** `(num % 5 == 0 || num % 7 == 0) && !(num % 5 == 0 && num % 7 == 0)`
- **หารลงตัวทั้งคู่ (AND):** `num % 5 == 0 && num % 7 == 0` หรือ `num % 35 == 0`
- **เปลี่ยนจำนวนต่อบรรทัด:** เปลี่ยนเป็น 5 ตัวต่อบรรทัดด้วย `count % 5 == 0`

⚠️ จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)

เข้าใจผิดว่าเครื่องหมาย ^ คือการยกกำลัง (เช่น 2^3) ในภาษา Java เครื่องหมาย ^ คือ Bitwise Exclusive OR เท่านั้น หากต้องการยกกำลังต้องเรียก `Math.pow(base, exp)`

ข้อที่ 6: Ex 6: เกมทายตัวเลขสุ่ม (Random Number & Guessing Loop)

Math.random & Binary Search Logic | `Lab06A_WhileLoop/GuessNumber.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

สร้างการสุ่มตัวเลขด้วย `Math.random()`, ควบคุมลูปการเล่นเกมจนกว่าผู้ใช้จะตอบถูก และให้คำใบ้ 'มากไป / น้อยไป'

ซอร์สโค้ดฉบับสมบูรณ์ (GuessNumber.java):

```
// SE101 - Lab 6A: Exercise 6 (GuessNumber.java)
import java.util.Scanner;

public class GuessNumber {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int secret = (int)(Math.random() * 100) + 1; // 1..100
        int guess = 0;
        int attempts = 0;

        while (guess != secret) {
            System.out.print("Guess a number (1-100): ");
            guess = scanner.nextInt();
            attempts++;

            if (guess > secret) {
                System.out.println("Too high!");
            } else if (guess < secret) {
                System.out.println("Too low!");
            } else {
                System.out.println("Correct! You got it in " + attempts + " attempts.");
            }
        }
        scanner.close();
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. สุ่มตัวเลขเป้าหมายในช่วง 0 ถึง 100: `int secret = (int)(Math.random() * 101);`
2. เปิดลูป `while (true)` เพื่อรับการทายจากผู้ใช้ซ้ำๆ
3. รับตัวเลขที่ทาย `int guess = sc.nextInt();`
4. เปรียบเทียบค่า:
   - ถ้า `guess > secret`: แสดง 'Too high'
   - ถ้า `guess < secret`: แสดง 'Too low'
   - ถ้าเท่ากัน: แสดง 'Yes, the number is ...' แล้ว `break;` จบลูป
5. นับจำนวนรอบที่ทายไปเพื่อบอกสถิติของผู้เล่น

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

```
ตัวอย่างการเล่น (สมมติเลขสุ่มได้ 42):
Guess a magic number between 0 and 100
Enter your guess: 50
Your guess is too high
Enter your guess: 25
Your guess is too low
```

```
Enter your guess: 37
Your guess is too low
Enter your guess: 42
Yes, the number is 42! (ทายสำเร็จใน 4 ครั้ง)
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **จำกัดจำนวนครั้ง:** ตั้ง `maxTries = 7` หากทายเกินให้แสดงว่าคุณแพ้ (You Lose) และเฉลยตัวเลข
- **สุ่มช่วง 1 ถึง 100:** `int secret = (int)(Math.random() * 100) + 1;`
- **ใช้ do-while:** เหมาะกับเกมทายตัวเลขเพราะต้องรับค่าทายอย่างน้อย 1 ครั้งแน่นอน

⚠️ จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)

ข้อผิดพลาดของการ Cast Type: ถ้าเขียน `(int)Math.random() * 101` วงเล็บจะ cast ค่า Math.random() ซึ่งมีค่าระหว่าง 0.0 ถึง 0.999... กลายเป็น 0 ก่อนเสมอ แล้วนำ 0 ไปคูณ 101 ผลลัพธ์จึงได้ 0 ตลอดกาล! วงเล็บที่ถูกต้องคือ `(int)(Math.random() * 101)`

ข้อที่ 7: Ex 7: เมนูโปรแกรมเครื่องคิดเลขซ้ำ (Menu-Driven Calculator)

Switch-Case & Input Validation `Lab06A_WhileLoop/Calculator.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

พัฒนาโปรแกรมจำลองเครื่องคิดเลขที่มีเมนูตัวเลือก วนรอบการทำงานซ้ำจนกว่าผู้ใช้จะเลือกเมนูออก (Quit) พร้อมป้องกันข้อผิดพลาดทางคณิตศาสตร์

ซอร์สโค้ดฉบับสมบูรณ์ (Calculator.java):

```
// SE101 - Lab 6A: Exercise 7 (Calculator.java)
import java.util.Scanner;

public class Calculator {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int choice = 0;

        System.out.println("=== Simple Calculator ===");

        while (choice != 5) {
            System.out.println("1. Add   2. Subtract   3. Multiply   4. Divide   5. Exit");
            System.out.print("Choose operation: ");
            choice = scanner.nextInt();

            switch (choice) {
                case 1:
                    System.out.print("Enter two numbers: ");
                    double add1 = scanner.nextDouble();
                    double add2 = scanner.nextDouble();
                    System.out.println(add1 + " + " + add2 + " = " + (add1 + add2));
                    break;
                case 2:
                    System.out.print("Enter two numbers: ");
                    double sub1 = scanner.nextDouble();
                    double sub2 = scanner.nextDouble();
                    System.out.println(sub1 + " - " + sub2 + " = " + (sub1 - sub2));
                    break;
                case 3:
                    System.out.print("Enter two numbers: ");
                    double mul1 = scanner.nextDouble();
                    double mul2 = scanner.nextDouble();
                    System.out.println(mul1 + " * " + mul2 + " = " + (mul1 * mul2));
                    break;
                case 4:
                    System.out.print("Enter two numbers: ");
                    double div1 = scanner.nextDouble();
                    double div2 = scanner.nextDouble();
                    if (div2 == 0) {
                        System.out.println("Error: Division by zero!");
                    } else {
                        System.out.println(div1 + " / " + div2 + " = " + (div1 / div2));
                    }
                    break;
                case 5:
                    System.out.println("Goodbye!");
                    break;
                default:
                    System.out.println("Invalid choice! Please choose 1-5.");
                    break;
```

```
            }
        }
        scanner.close();
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. แสดงเมนูทางเลือก 1: Add, 2: Subtract, 3: Multiply, 4: Divide, 5: Exit
2. ใช้ลูป `while (true)` หรือ `while (choice != 5)`
3. รับตัวเลือกเมนู `int choice = sc.nextInt();`
4. ถ้าเลือก 5: พิมพ์ข้อความอำลาและ `break;`
5. ถ้าเลือก 1-4: รับตัวเลข 2 จำนวน (num1, num2) แล้วส่งเข้า `switch (choice)`
6. **ตรวจสอบ Division by Zero:** ใน Case 4 หาก `num2 == 0` ต้องแจ้งเตือน Error ทันที ไม่ปล่อยให้โปรแกรมหารเลขเพราะจะทำให้โปรแกรมค้างหรือเกิด ArithmeticException

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

ตัวอย่างหน้าจอการทำงาน:

```
--- Simple Calculator ---
1. Add (+)
2. Subtract (-)
3. Multiply (*)
4. Divide (/)
5. Exit
Enter choice (1-5): 4
Enter first number: 10
Enter second number: 0
Error: Division by zero is not allowed!

Enter choice (1-5): 1
Enter first number: 15
Enter second number: 27
Result: 15.0 + 27.0 = 42.0

Enter choice (1-5): 5
Exiting program... Goodbye!
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **เพิ่มเมนูยกกำลังหรือหารเอาเศษ:** Case 5: Modulo (%), Case 6: Power (Math.pow)
- **รับเครื่องหมายทางคณิตศาสตร์เป็นตัวอักษร:** เช่น รับ '+', '-', '*', '/' แทนตัวเลข 1, 2, 3, 4
- **ทำเป็น Method ย่อย:** แยกฟังก์ชัน `add(a,b)`, `subtract(a,b)`, `divide(a,b)`

⚠️ จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)

ลืมใส่คำสั่ง `break;` ท้ายแต่ละ case ใน switch-case ทำให้อัลกอริทึมไหลทะลุไปคำนวณเคสอื่นๆ ด้วย และลืมดักจับกรณีผู้ใช้ป้อนตัวเลือกนอกเหนือจาก 1-5 (ควรมี `default:` รองรับ)

บทที่ 2: For Loops & Nested Loops (Lab 07A — 7 Exercises)

ภาพรวมเนื้อหา: ลูปตัวแปรคู่, Break/Continue, อัลกอริทึม GCD, การหาจำนวนเฉพาะ, และพีระมิดเลขยกกำลังฐานสอง

ข้อที่ 8: Ex 1: โครงสร้างพื้นฐานของ For Loop และการคำนวณกำลังสอง

Basic For Loop Structure `Lab07A_ForLoop/Exercise1.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

เข้าใจ 3 ส่วนประกอบหลักของ For Loop ได้แก่ Initialization (กำหนดค่าเริ่มต้น), Loop Continuation Condition (เงื่อนไขการวนรอบ), และ Update Expression (การเปลี่ยนแปลงค่า)

ซอร์สโค้ดฉบับสมบูรณ์ (Exercise1.java):

```
public class Exercise1 {
    public static void main(String[] args) {
        // Step 1: เริ่มต้น loop โดยให้ i = 1
        // Step 2: ตรวจสอบเงื่อนไข i <= 4 ถ้าเป็นจริงจะเข้าทำงานใน loop
        // Step 3: ทำงานจบ 1 รอบ จะสั่ง i++ (เพิ่มค่า i ทีละ 1)
        for (int i = 1; i <= 4; i++) {
            System.out.println(i + " squared = " + (i * i));
            }
        System.out.println("Done");
        }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. หัวลูป `for (int i = 1; i <= 4; i++)` ประกอบด้วย:
   1. `int i = 1;` กำหนดตัวแปรนับ i เริ่มที่ 1
   2. `i <= 4;` วนซ้ำตราบเท่าที่ i ไม่เกิน 4
   3. `i++` เพิ่มค่า i ทีละ 1 หลังจบรอบ
2. ภายในลูป: คำนวณค่ากำลังสอง `i * i` แล้วแสดงผลด้วย `System.out.println(i + " squared = " + (i * i));`
3. เมื่อ i กลายเป็น 5 เงื่อนไข `5 <= 4` เป็นเท็จ จึงหลุดออกจากลูป
4. หลังจบลูป: พิมพ์คำว่า 'Done!'

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

ลำดับการวนรอบและผลลัพธ์:

```
i = 1 -> 1 <= 4 (T) -> พิมพ์ "1 squared = 1"  -> i เพิ่มเป็น 2
i = 2 -> 2 <= 4 (T) -> พิมพ์ "2 squared = 4"  -> i เพิ่มเป็น 3
i = 3 -> 3 <= 4 (T) -> พิมพ์ "3 squared = 9"  -> i เพิ่มเป็น 4
i = 4 -> 4 <= 4 (T) -> พิมพ์ "4 squared = 16" -> i เพิ่มเป็น 5
i = 5 -> 5 <= 4 (F) -> หลุดลูป
พิมพ์ "Done!" ทางจอภาพ
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **เขียนเทียบเท่าในรูป While Loop:**
  `int i = 1; while (i <= 4) { System.out.println(i + " squared = " + (i * i)); i++; }`
- **ยกกำลังสาม (Cube):** `i * i * i` หรือ `(int)Math.pow(i, 3)`
- **รับจำนวนรอบจากผู้ใช้:** เปลี่ยน 4 เป็นตัวแปร n ที่รับจาก `sc.nextInt()`

⚠️ จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)

ระวังการใส่เครื่องหมาย semicolon หลังหัว for เช่น `for (int i = 1; i <= 4; i++);` จะทำให้ลูปทำงานเป็นคำสั่งว่าง (Empty Statement) ตัวแปร i จะนับจนถึง 5 แล้วค่อยไปทำปีกกาด้านล่างเพียงรอบเดียว ซึ่งเป็น Logic Error ที่คอมไพเลอร์ไม่แจ้งเตือน

ข้อที่ 9: Ex 2: การคำนวณผลรวมสะสม แยกผลรวมเลขคู่และเลขคี่ (Even/Odd Accumulators)

Multiple Accumulators & Parity Check `Lab07A_ForLoop/SumNumber.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

ฝึกการใช้ตัวแปรสะสมค่า (Accumulators) หลายตัวพร้อมกันในลูปเดียว เพื่อคำนวณผลรวมทั้งหมด, ผลรวมเฉพาะเลขคู่, และผลรวมเฉพาะเลขคี่

ซอร์สโค้ดฉบับสมบูรณ์ (SumNumber.java):

```
public class SumNumber {
    public static void main(String[] args) {
        int sum = 0, evenSum = 0, oddSum = 0;
        for (int i = 1; i <= 100; i++) {
            sum += i;
            if (i % 2 == 0)
                evenSum += i;
            else
                oddSum += i;
        }
        System.out.println("Sum 1-100 = " + sum);
        System.out.println("Sum of even numbers = " + evenSum);
        System.out.println("Sum of odd numbers  = " + oddSum);
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. ประกาศตัวแปรสะสมค่า 3 ตัวเริ่มต้นเป็น 0:
   `int totalSum = 0;`
   `int evenSum = 0;`
   `int oddSum = 0;`
2. วนลูป `for (int i = 1; i <= 100; i++)` ตรวจสอบเลข 1 ถึง 100
3. ทุกรอบบวกเข้าผลรวมทั้งหมด: `totalSum += i;`
4. ตรวจสอบเงื่อนไขเลขคู่/คี่:
   - ถ้า `i % 2 == 0`: สะสมเข้า `evenSum += i;`
   - ถ้าไม่ใช่: สะสมเข้า `oddSum += i;`
5. เมื่อจบลูป แสดงผลลัพธ์ทั้ง 3 บรรทัด

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

```
ตัวอย่างผลการคำนวณ 1 ถึง 100:
Sum of numbers from 1 to 100 = 5050
Sum of even numbers from 1 to 100 = 2550
Sum of odd numbers from 1 to 100 = 2500

การตรวจสอบความถูกต้องทางคณิตศาสตร์:
totalSum ต้องเท่ากับ evenSum + oddSum พอดี
2550 + 2500 = 5050 (ถูกต้อง 100%)
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **ใช้สูตรคณิตศาสตร์หา Total Sum:** $N(N + 1) / 2 = 100 \times 101 / 2 = 5050$

- **แยก 2 ลูปแบบก้าวข้าม (Step by 2):**
  - คู่: `for (int i = 2; i <= 100; i += 2) evenSum += i;`
  - คี่: `for (int i = 1; i <= 100; i += 2) oddSum += i;`
- **หาค่าเฉลี่ยของเลขคู่และเลขคี่เพิ่มเติม:** นับ `evenCount` และ `oddCount` แล้วนำไปหาร

⚠️ จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)

ลืมกำหนดค่าเริ่มต้นเป็น 0 ให้กับตัวแปรสะสม (เช่น ประกาศ `int sum;` เฉยๆ) คอมไพเลอร์ของ Java จะไม่อนุญาตให้ใช้ตัวแปรที่ยังไม่ได้ initialize ในการบวกสะสม (The local variable may not have been initialized)

ข้อที่ 10: Ex 3: ตารางแปลงหน่วยกิโลกรัม-ปอนด์ ด้วยลูปตัวแปรคู่ (Two-Variable For Loop)

Multi-variable Loop & Formatted Table `Lab07A_ForLoop/UnitTable.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

เรียนรู้เทคนิคขั้นสูงของคำสั่ง for ที่สามารถประกาศและปรับเปลี่ยนค่าของตัวแปร 2 ตัวพร้อมกันในหัวลูปเดียว (Parallel Iteration)

ซอร์สโค้ดฉบับสมบูรณ์ (UnitTable.java):

```
public class UnitTable {
    public static void main(String[] args) {
        System.out.println("Kilograms    Pounds");
        for (int kg = 1, lb = 20; kg <= 19; kg += 2, lb += 5) {
            System.out.printf("%-12d%.2f%n", kg, (double) lb);
        }
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. พิมพ์หัวตารางเปรียบเทียบ: Kilograms vs Pounds
2. หัวลูปกำหนดตัวแปร 2 ตัวพร้อมกัน:
   `for (int kg = 1, lb = 20; kg <= 199; kg += 2, lb += 5)`
   - kg เริ่มจาก 1 เพิ่มขึ้นรอบละ 2 (1, 3, 5, ..., 199)
   - lb เริ่มจาก 20 เพิ่มขึ้นรอบละ 5 (20, 25, 30, ...)
   - ใช้เครื่องหมายจุลภาค (,) คั่นระหว่างตัวแปร
3. แปลงหน่วย: $1\text{ kg} = 2.2\text{ lb}$ และ $1\text{ lb} = 0.453592\text{ kg}$
4. จัดหน้าตาตาราง 4 คอลัมน์ด้วย `printf("%-10d%-10.1f | %-10d%-10.2f\n", kg, kg * 2.2, lb, lb * 0.4545)`

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

หน้าจอผลลัพธ์ของโปรแกรม:

```
Kilograms    Pounds      |   Pounds      Kilograms
------------------------------------------------
1            2.2         |   20          9.09
3            6.6         |   25          11.36
5            11.0        |   30          13.64
...
197          433.4       |   510         231.82
199          437.8       |   515         234.09
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **เขียนโดยใช้ตัวแปรเดียว:** วนเฉพาะ kg แล้วคำนวณ lb ตามรอบ: `int lb = 20 + ((kg - 1) / 2) * 5;`
- **เขียนในรูป While Loop:** ต้องประกาศ kg, lb แยกบรรทัดก่อนลูป แล้วเพิ่มค่า kg += 2, lb += 5 ท้ายลูป
- **แปลงหน่วยอุณหภูมิคู่ขนาน:** เซลเซียสคู่กับฟาเรนไฮต์

⚠️ จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)

ใส่เครื่องหมาย semicolon (;) แทนเครื่องหมาย comma (,) ในหัว for เช่น `for (int kg = 1; lb = 20; ...)` ใน Java อนุญาตให้มีเครื่องหมาย semicolon ในหัว for ได้เพียง 2 ตัวเท่านั้นเพื่อแยก 3 ส่วนหลัก หากต้องการหลายตัวแปรต้องใช้ comma (,) คั่น

ข้อที่ 11: Ex 4: การควบคุมวงจรลูปด้วย Break และ Continue

Loop Control Jump Statements `Lab07A_ForLoop/BreakContinue.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

เข้าใจความแตกต่างอย่างชัดเจนระหว่างคำสั่ง `break` (หยุดและออกจากลูปทันที) และคำสั่ง `continue` (ข้ามรอบปัจจุบันไปเริ่มรอบถัดไป)

ซอร์สโค้ดฉบับสมบูรณ์ (BreakContinue.java):

```
public class BreakContinue {
    public static void main(String[] args) {
        System.out.print("Snippet A: ");
        for (int i = 1; i <= 10; i++) {
            if (i == 6) break;
            System.out.print(i + " ");
        }

        System.out.println();

        System.out.print("Snippet B: ");
        for (int i = 1; i <= 10; i++) {
            if (i % 3 == 0) continue;
            System.out.print(i + " ");
        }
        System.out.println();
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. **Snippet A (คำสั่ง break):**
   - วนลูป `for (int i = 1; i <= 10; i++)`
   - ตรวจสอบ `if (i == 6) break;`
   - ทันทีที่ i มีค่าเป็น 6 คำสั่ง break จะบังคับให้โปรแกรมกระโดดออกจากลูปทันที ส่งผลให้เลข 6 ถึง 10 ไม่ถูกพิมพ์
2. **Snippet B (คำสั่ง continue):**
   - วนลูป `for (int i = 1; i <= 10; i++)`
   - ตรวจสอบ `if (i % 3 == 0) continue;`
   - ถ้าตัวเลขหารด้วย 3 ลงตัว (เลข 3, 6, 9) คำสั่ง continue จะข้ามคำสั่งพิมพ์ด้านล่าง แล้วกระโดดไปทำ `i++` เพื่อเริ่มรอบใหม่ทันที

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

เปรียบเทียบผลลัพธ์ของทั้งสองชุดคำสั่ง:

```
[Snippet A: break at 6]
i = 1 -> พิมพ์ 1
i = 2 -> พิมพ์ 2
i = 3 -> พิมพ์ 3
i = 4 -> พิมพ์ 4
i = 5 -> พิมพ์ 5
i = 6 -> เข้าเงื่อนไข break -> จบลูปทันที!
Output: 1 2 3 4 5

[Snippet B: continue at multiple of 3]
พิมพ์เลข: 1, 2, (3 ข้าม), 4, 5, (6 ข้าม), 7, 8, (9 ข้าม), 10
Output: 1 2 4 5 7 8 10
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **Labeled Break ใน Nested Loop:** `outerLoop: for (...) { for (...) { if (...) break outerLoop; } }` เพื่อกระโดดออกจากลูปซ้อนทั้งหมดในคำสั่งเดียว
- **ใช้ตัวแปร Boolean Flag แทน break:** `boolean keepRunning = true; while (keepRunning) { ... if (...) keepRunning = false; }`
- **กรองข้อมูล:** ใช้ continue ในการกรองข้อมูลที่ไม่ต้องการ เช่น ข้ามค่าที่ติดลบ

⚠️ จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)

เขียนคำสั่งวางไว้ใต้ break หรือ continue ในบล็อกเดียวกันโดยไม่มีเงื่อนไขครอบ จะเกิดข้อผิดพลาด `Unreachable code` เพราะคอมไพเลอร์รู้ว่าคำสั่งบรรทัดนั้นไม่มีทางถูกเรียกทำงานได้จริง

ข้อที่ 12: Ex 5: การหาตัวหารร่วมมาก (Greatest Common Divisor - GCD)

Brute Force Search Algorithm `Lab07A_ForLoop/GCD.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

สร้างอัลกอริทึมค้นหาตัวหารร่วมมากของจำนวนเต็มบวก 2 จำนวน ด้วยวิธี Brute-force Search จากล่างขึ้นบน

ซอร์สโค้ดฉบับสมบูรณ์ (GCD.java):

```
import java.util.Scanner;

public class GCD {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Enter n1: ");
        int n1 = input.nextInt();
        System.out.print("Enter n2: ");
        int n2 = input.nextInt();

        int gcd = 1;

        for (int k = 2; k <= n1 && k <= n2; k++) {
            if (n1 % k == 0 && n2 % k == 0) {
                gcd = k;
            }
        }

        System.out.println("GCD(" + n1 + ", " + n2 + ") = " + gcd);
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. รับจำนวนเต็มบวก 2 ตัว n1 และ n2 จากผู้ใช้
2. กำหนดค่าเริ่มต้นของ ห.ร.ม. `int gcd = 1;` (เพราะจำนวนเต็มบวกใดๆ ย่อมมี 1 เป็นตัวหารร่วมเสมอ)
3. หาขอบเขตสูงสุดที่จะทดสอบ: ตัวหารร่วมไม่มีทางเกินค่าน้อยของสองตัว จึงเริ่มลูป `int k = 2; k <= n1 && k <= n2; k++`
4. ตรวจสอบเงื่อนไขการหารลงตัวทั้งคู่: `if (n1 % k == 0 && n2 % k == 0)`
5. หาก k หารทั้งสองจำนวนลงตัว ให้อัปเดต `gcd = k;` (ค่า k ที่มากขึ้นเรื่อยๆ จะทับค่าเดิมจนได้ค่าที่มากที่สุด)
6. เมื่อจบลูป แสดงผลค่า gcd ที่ได้

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

```
ตัวอย่างการหา GCD ของ n1 = 24 และ n2 = 36:
เริ่มต้น: gcd = 1, ทดสอบ k = 2 ถึง 24
- k = 2: 24%2==0 และ 36%2==0 -> จริง -> gcd = 2
- k = 3: 24%3==0 และ 36%3==0 -> จริง -> gcd = 3
- k = 4: 24%4==0 และ 36%4==0 -> จริง -> gcd = 4
- k = 6: 24%6==0 และ 36%6==0 -> จริง -> gcd = 6
- k = 12: 24%12==0 และ 36%12==0 -> จริง -> gcd = 12
... ไม่พบตัวหารร่วมที่มากกว่านี้
ผลลัพธ์: GCD of 24 and 36 is 12
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **ค้นหาถอยหลังจากบนลงล่าง (Fast Brute-force):** เริ่มจาก `Math.min(n1, n2)` ลงมาหา 1 ตัวแรกที่หารลงทั้งคู่คือ ห.ร.ม. ทันที แล้วสั่ง break ได้เลย
- **Euclidean Algorithm (วิธีของยุคลิดด้วย Modulo):** มีประสิทธิภาพสูงสุด $O(\log N)$
  `while (n2 != 0) { int temp = n2; n2 = n1 % n2; n1 = temp; } return n1;`
- **หา ค.ร.น. (LCM):** ใช้สูตร $LCM(a, b) = (a \times b) / GCD(a, b)$

⚠️ จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)

ตั้งค่าเริ่มต้น `gcd = 0` หากป้อนตัวเลขที่เป็น Coprime กัน (เช่น 7 และ 9) ลูปจะไม่เจอตัวหารตั้งแต่ 2 ขึ้นไป ส่งผลให้โปรแกรมตอบว่า gcd เป็น 0 ซึ่งผิดหลักคณิตศาสตร์ ค่าเริ่มต้นต้องเป็น **1** เสมอ

ข้อที่ 13: Ex 6: การพิมพ์จำนวนเฉพาะ 50 ตัวแรก (Prime Numbers & Primality Test)

Nested Loop & Primality Testing `Lab07A_ForLoop/PrimeNumbers.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

สร้าง Nested Loops เพื่อค้นหาและแสดงจำนวนเฉพาะ (Prime Numbers) 50 ตัวแรก โดยจัดแถวละ 10 จำนวนอย่างเป็นระเบียบ

ซอร์สโค้ดฉบับสมบูรณ์ (PrimeNumbers.java):

```
public class PrimeNumbers {
    public static void main(String[] args) {
        int count = 0;
        int number = 2;

        while (count < 50) {
            boolean isPrime = true;

            for (int divisor = 2; divisor <= number / 2; divisor++) {
                if (number % divisor == 0) {
                    isPrime = false;
                    break;
                }
            }

            if (isPrime) {
                System.out.print(number + " ");
                count++;

                if (count % 10 == 0) {
                    System.out.println();
                }
            }

            number++;
        }
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. กำหนดตัวแปร: `int count = 0;` (นับจำนวนเฉพาะที่พบ), `int number = 2;` (ตัวเลขแรกที่จะทดสอบ)
2. ลูปนอก `while (count < 50)`: ทำงานตราบใดที่ยังพบจำนวนเฉพาะไม่ครบ 50 ตัว
3. สมมติให้ตัวเลขนั้นเป็นจำนวนเฉพาะไว้ก่อน: `boolean isPrime = true;`
4. ลูปใน: วนทดสอบตัวหาร `for (int divisor = 2; divisor <= number / 2; divisor++)`
5. หาก `number % divisor == 0`: แปลว่ามีตัวหารลงตัว แสดงว่าไม่ใช่จำนวนเฉพาะ ให้ตั้ง `isPrime = false; break;`
6. ตรวจสอบหลังลูปใน: ถ้า `isPrime == true` ให้นับเพิ่ม `count++;` แล้วพิมพ์
7. ตัดบรรทัดใหม่ทุก 10 ตัว: `if (count % 10 == 0) System.out.println();`
8. เพิ่ม `number++;` เพื่อทดสอบตัวเลขถัดไป

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

```
ขั้นตอนการทดสอบตัวเลข:
- number = 2: divisor ลูปไม่ทำงาน -> isPrime=true -> count=1, พิมพ์ 2
- number = 3: divisor ลูปไม่ทำงาน -> isPrime=true -> count=2, พิมพ์ 3
```

```
- number = 4: divisor=2 -> 4%2==0 -> isPrime=false, break -> ไม่พิมพ์
- number = 5: divisor=2 -> 5%2!=0 -> isPrime=true -> count=3, พิมพ์ 5

ผลลัพธ์หน้าจอ (10 ตัวต่อบรรทัด):
  2   3   5   7  11  13  17  19  23  29
 31  37  41  43  47  53  59  61  67  71
 73  79  83  89  97 101 103 107 109 113
127 131 137 139 149 151 157 163 167 173
179 181 191 193 197 199 211 223 227 229
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **เพิ่มความเร็วด้วย Math.sqrt:** เปลี่ยนขอบเขตตัวหารเป็น `divisor <= Math.sqrt(number)`
- **แยก Primality Test เป็น Method ย่อย:** `public static boolean isPrime(int n)` (ดูใน Lab 10B)
- **หาจำนวนเฉพาะในช่วง [A, B]:** เปลี่ยนลูปนอกเป็น `for (int n = start; n <= end; n++)`

⚠️ จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)

จุดผิดยอดฮิต: ประกาศตัวแปร `boolean isPrime = true;` ไว้นอกลูป while ส่งผลให้ทันทีที่พบเลขที่ไม่ใช่จำนวนเฉพาะแม้แต่ตัวเดียว ตัวแปร isPrime จะกลายเป็น false ตลอดไป และโปรแกรมจะไม่พิมพ์จำนวนเฉพาะตัวใดออกมาอีกเลย! ต้องรีเซ็ต `isPrime = true;` *ภายใน* ลูปนอกทุกรอบ

ข้อที่ 14: Ex 7: พีระมิดเลขยกกำลังฐาน 2 กึ่งกลางหน้าจอ (Powers of 2 Centered Pyramid)

Nested Loops & Geometric Pattern Printing `Lab07A_ForLoop/Pyramid.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

สร้างลูปซ้อน 3 ชั้น (Nested Loops) เพื่อจัดตำแหน่งช่องว่างและพิมพ์ตัวเลขยกกำลังของ 2 ($2^0, 2^1, ..., 2^k$) ในรูปทรงพีระมิดหน้าจั่วกึ่งกลางหน้าจออย่างสมบูรณ์แบบ

ซอร์สโค้ดฉบับสมบูรณ์ (Pyramid.java):

```
public class Pyramid {
    public static void main(String[] args) {
        int totalRows = 7;

        for (int row = 1; row <= totalRows; row++) {
            for (int space = 1; space <= totalRows - row; space++) {
                System.out.print("  ");
            }

            for (int col = 0; col < row; col++) {
                int value = (int) Math.pow(2, col);
                System.out.printf("%4d", value);
            }

            System.out.println();
        }
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. กำหนดความสูงของพีระมิด `int totalRows = 7;` (หรือ 8)
2. **ลูปชั้นนอก (Outer Loop):** `for (int row = 1; row <= totalRows; row++)` ควบคุมแถวตั้งแต่ 1 ถึง 7
3. **ลูปชั้นในที่ 1 (Leading Spaces):** `for (int space = 1; space <= (totalRows - row) * 2; space++)` `System.out.print("  ");` พิมพ์ช่องว่างเยื้องซ้ายเพื่อดันตัวเลขให้เป็นทรงสามเหลี่ยมพีระมิดกึ่งกลาง
4. **ลูปชั้นในที่ 2 (Numbers):** `for (int col = 0; col < row; col++)`
   - คำนวณเลขยกกำลังฐานสอง: `(int)Math.pow(2, col)`
   - จัดรูปแบบตัวเลขให้กว้าง 4 ช่องพอดีเสมอด้วย `System.out.printf("%4d", value);`
5. ขึ้นบรรทัดใหม่เมื่อพิมพ์จบแถว: `System.out.println();`

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

โครงสร้างการคำนวณแต่ละแถว:

```
Row 1: (7-1)*2 = 12 spaces | pow(2,0) = 1
Row 2: (7-2)*2 = 10 spaces | pow(2,0)=1, pow(2,1)=2
Row 3: (7-3)*2 =  8 spaces | pow(2,0)=1, pow(2,1)=2, pow(2,2)=4
...
Row 7: (7-7)*2 =  0 spaces | 1   2   4   8  16  32  64
```

รูปทรง Output ที่ถูกต้องตามโจทย์ (Centered Pyramid):

```
                    1
                  1   2
                1   2   4
              1   2   4   8
            1   2   4   8  16
```

```
     1   2   4   8  16  32
1    2   4   8  16  32  64
```

🔄 **แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)**

- **พีระมิดสมมาตรแบบเพิ่มแล้วลด (Palindrome Pyramid):** พิมพ์ 1 2 4 8 4 2 1 ในแถวเดียว โดยใช้ลูปเลขขาขึ้นแล้วตามด้วยลูปเลขขาลง
- **พีระมิดดาว (*):** แทนที่ตัวเลขด้วยเครื่องหมายดอกจัน *
- **สามเหลี่ยมชิดซ้าย (Left-aligned):** ตัดลูปช่องว่าง (Spaces Loop) ทิ้งไป จะได้สามเหลี่ยมมุมฉาก

⚠️ **จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)**

จุดที่ห้ามทำผิด: 1. ใช้เครื่องหมาย ^ แทนยกกำลัง (เช่น `2 ^ col`) ซึ่งผิด เพราะใน Java ต้องใช้ `(int)Math.pow(2, col)`
2. ลืมหรือคำนวณจำนวนช่องว่างผิด ทำให้ตัวเลขเอียงกลายเป็นสามเหลี่ยมมุมฉาก ไม่ใช่พีระมิดหน้าจั่วกึ่งกลาง
3. ไม่ใช้ %4d ทำให้เมื่อตัวเลขหลักสิบ (16, 32, 64) โผล่มาแล้วตำแหน่งของคอลัมน์จะเบี้ยวทันที

บทที่ 3: Methods Part 1: Value-Returning & Math Methods (Lab 10A — 5 Exercises)

ภาพรวมเนื้อหา: การประกาศ Method คืนค่า, Parameters, การเรียกใช้ซ้อน (Nested Calls), และการจำแนกรูปทรงเรขาคณิต

ข้อที่ 15: Ex 1-2: การสร้าง Method คืนค่า และการเรียกใช้แบบซ้อน (Nested Method Calls)

Value-Returning Method & Method Chaining | `Lab10A_Methods1/TestMax.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

เรียนรู้โครงสร้างการประกาศฟังก์ชันที่ส่งค่ากลับ (Value-returning method), การส่งผ่าน Parameters, และเทคนิคการเรียก Method ซ้อนกันเพื่อหาค่ามากสุดในหลายตัวแปร

ซอร์สโค้ดฉบับสมบูรณ์ (TestMax.java):

```
// SE101 - Lab 10A: Exercise 1 & 2 (TestMax.java)
public class TestMax {
    public static void main(String[] args) {
        int i = 5;
        int j = 2;
        int x = 9;
        int y = 7;

        // Exercise 1: Max between two numbers
        int k = max(i, j);
        System.out.println("Max = " + k);

        // Exercise 2: Max among four numbers using nested max() calls
        int max4 = max(max(i, j), max(x, y));
        System.out.println("The maximum between " + i + ", " + j + ", " + x + " and " + y + " is " +
max4);
    }

    public static int max(int num1, int num2) {
        int result;
        if (num1 > num2) {
            result = num1;
        } else {
            result = num2;
        }
        return result;
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. สร้าง Method `public static int max(int num1, int num2)`
2. ภายใน Method: ใช้เงื่อนไข `if (num1 > num2) return num1; else return num2;`
3. ใน `main()`: ทดสอบเรียกใช้ด้วยตัวเลข 2 ตัว `int k = max(i, j);`
4. เทคนิค **Nested Method Calls** หาค่ามากสุดจาก 4 ตัวแปร (i, j, k, l):
   `int largest = max(max(i, j), max(k, l));`
   - Java จะคำนวณ `max(i, j)` ก่อน
   - ตามด้วยคำนวณ `max(k, l)`
   - แล้วนำผลลัพธ์ของทั้งสองคู่มาหา `max()` อีกครั้งในรอบสุดท้าย

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

```
ตัวอย่างการหา Max ของ 4 ตัวแปร (i = 5, j = 2, k = 14, l = 9):
1. คำนวณ max(5, 2)   -> คืนค่า 5
2. คำนวณ max(14, 9)  -> คืนค่า 14
```

```
3. คำนวณ max(5, 14)  -> คืนค่า 14
ผลลัพธ์: The largest number is 14
```

🔄 **แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)**

- **เขียนสั้นด้วย Ternary:** `public static int max(int a, int b) { return (a > b) ? a : b; }`
- **Method Overloading สำหรับชนิด double:** ประกาศ `public static double max(double a, double b)` ชื่อเหมือนกันแต่ parameter ต่างชนิดกัน
- **หาค่าต่ำสุด (Min):** เปลี่ยนเครื่องหมาย > เป็น <

⚠️ **จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)**

เขียนคำสั่ง `return` ไม่ครอบคลุมทุกกิ่งก้านของเงื่อนไข เช่น ใส่ return เฉพาะใน if แต่ลืม else ทำให้คอมไพเลอร์แจ้งเตือน `Missing return statement`

ข้อที่ 16: Ex 3: ฟังก์ชันคำนวณผลรวมในช่วงจำนวนเต็ม (Sum of Range Method)

Method with Parameters & Accumulator | `Lab10A_Methods1/SumRange.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

ฝึกการแยกฟังก์ชันคำนวณผลรวมตัวเลขจากขอบเขตเริ่มต้นถึงสิ้นสุด เพื่อให้โค้ดส่วนคำนวณสามารถนำกลับมาใช้ซ้ำ (Reusable) ได้หลายครั้ง

ซอร์สโค้ดฉบับสมบูรณ์ (SumRange.java):

```
// SE101 - Lab 10A: Exercise 3 (SumRange.java)
public class SumRange {
    public static void main(String[] args) {
        System.out.println("Sum from 1 to 10 is " + sum(1, 10));
        System.out.println("Sum from 20 to 30 is " + sum(20, 30));
        System.out.println("Sum from 35 to 45 is " + sum(35, 45));
    }

    public static int sum(int i1, int i2) {
        int total = 0;
        for (int i = i1; i <= i2; i++) {
            total += i;
        }
        return total;
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. ประกาศ Method: `public static int sum(int i1, int i2)` รับขอบเขตล่าง `i1` และขอบเขตบน `i2`
2. ภายใน Method: สร้างตัวแปร `int result = 0;`
3. วนลูปสะสมผลรวม: `for (int i = i1; i <= i2; i++) result += i;`
4. ส่งผลลัพธ์กลับ: `return result;`
5. ใน `main()`: เรียกใช้งานฟังก์ชันซ้ำ 3 ครั้งในช่วงต่างๆ ได้แก่ 1 ถึง 10, 20 ถึง 37, และ 35 ถึง 49

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

ผลลัพธ์การเรียกใช้งานแต่ละช่วง:

```
- sum(1, 10):  1 + 2 + ... + 10 = 55
- sum(20, 37): 20 + 21 + ... + 37 = 513
- sum(35, 49): 35 + 36 + ... + 49 = 630

หน้าจอ Output:
Sum from 1 to 10 is 55
Sum from 20 to 37 is 513
Sum from 35 to 49 is 630
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **กรณีขอบเขตกลับด้าน (i1 > i2):** เพิ่มการสลับค่าก่อนวนลูป:
  `if (i1 > i2) { int t = i1; i1 = i2; i2 = t; }`
- **คำนวณด้วยสูตรคณิตศาสตร์:** $(i1 + i2) \times (i2 - i1 + 1) / 2$ ทำงานเร็วในเวลา $O(1)$

- **หาผลรวมเฉพาะเลขคู่ในช่วง:** เพิ่มเงื่อนไข `if (i % 2 == 0)` ภายในลูป

⚠️ จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)

กำหนดเงื่อนไขลูปเป็น `i < i2` แทนที่จะเป็น `i <= i2` ทำให้ตัวเลขตัวสุดท้าย (i2) ไม่ถูกนำไปบวกด้วย เกิดข้อผิดพลาด Off-by-one

ข้อที่ 17: Ex 4: ฟังก์ชันแปลงอุณหภูมิสองทาง (Two-Way Temperature Converter)

Mathematical Formulas & Floating-Point Methods | `Lab10A_Methods1/TempConverter.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

สร้างฟังก์ชันคณิตศาสตร์แปลงเซลเซียสเป็นฟาเรนไฮต์ และฟาเรนไฮต์เป็นเซลเซียส พร้อมพิมพ์ตารางเปรียบเทียบค่าคู่ขนาน

ซอร์สโค้ดฉบับสมบูรณ์ (TempConverter.java):

```
// SE101 - Lab 10A: Exercise 4 (TempConverter.java)
import java.util.Scanner;

public class TempConverter {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String continueChoice;

        do {
            System.out.print("Convert (C/F): ");
            String choice = scanner.next().trim().toUpperCase();

            System.out.print("Enter temperature: ");
            double temp = scanner.nextDouble();

            if (choice.equals("C")) {
                double fahrenheit = celsiusToFahrenheit(temp);
                System.out.printf("%.1f °C = %.2f °F%n", temp, fahrenheit);
            } else if (choice.equals("F")) {
                double celsius = fahrenheitToCelsius(temp);
                System.out.printf("%.1f °F = %.2f °C%n", temp, celsius);
            } else {
                System.out.println("Invalid choice! Choose C or F.");
            }

            System.out.print("Another conversion? (yes/no): ");
            continueChoice = scanner.next().trim().toLowerCase();

        } while (continueChoice.equals("yes") || continueChoice.equals("y"));

        System.out.println("Goodbye!");
        scanner.close();
    }

    public static double celsiusToFahrenheit(double c) {
        return (9.0 / 5) * c + 32;
    }

    public static double fahrenheitToCelsius(double f) {
        return (5.0 / 9) * (f - 32);
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. Method แปลง C เป็น F: `public static double celsiusToFahrenheit(double c)`
   สูตร: `return (9.0 / 5) * c + 32;`
2. Method แปลง F เป็น C: `public static double fahrenheitToCelsius(double f)`
   สูตร: `return (5.0 / 9) * (f - 32);`

3. ใน `main()`: วนลูป `for (double c = 40.0, f = 120.0; c >= 31.0; c--, f -= 10)`

4. เรียกฟังก์ชันแปลงค่าทั้งสองฝั่ง และจัดรูปแบบตารางด้วย `printf`

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

```
ผลการแปลงอุณหภูมิในตาราง:
Celsius    Fahrenheit  |  Fahrenheit  Celsius
-----------------------------------------------
40.0       104.00      |  120.0       48.89
39.0       102.20      |  110.0       43.33
38.0       100.40      |  100.0       37.78
...
32.0       89.60       |   40.0        4.44
31.0       87.80       |   30.0       -1.11
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **แปลงเป็นเคลวิน (Kelvin):** $K = C + 273.15$
- **รับค่าอุณหภูมิจากผู้ใช้:** ใช้ Scanner ให้ผู้ใช้เลือกโหมด 1: C->F หรือ 2: F->C
- **เพิ่มข้อความเตือนความร้อน:** เช็คว่าหาก $F > 100$ ให้พิมพ์ '(Warning: Hot)'

⚠️ จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)

ข้อผิดพลาดร้ายแรงที่สุด: เขียนสูตรเป็น `(9 / 5) * c + 32` โดยไม่ใส่ทศนิยม `.0` ใน Java `9 / 5` เป็นการหารจำนวนเต็ม ได้ผลลัพธ์เป็น **1** เสมอ ทำให้สูตรกลายเป็น $1 \times c + 32$ ซึ่งผิดทั้งหมด! ต้องเขียน `(9.0 / 5)` หรือ `(9.0 / 5.0)`

ข้อที่ 18: Ex 5: การคำนวณระยะห่างระหว่างจุด 2 มิติ (Euclidean Distance)

Math Library & Multi-parameter Method | `Lab10A_Methods1/Distance.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

สร้างฟังก์ชันรับพิกัด 4 จุด (x_1, y_1) และ (x_2, y_2) แล้วคำนวณระยะทางแบบยูคลิดโดยใช้ฟังก์ชันจากคลาส `Math`

ซอร์สโค้ดฉบับสมบูรณ์ (Distance.java):

```
// SE101 - Lab 10A: Exercise 5 (Distance.java)
import java.util.Scanner;

public class Distance {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter point 1 (x y): ");
        double x1 = scanner.nextDouble();
        double y1 = scanner.nextDouble();

        System.out.print("Enter point 2 (x y): ");
        double x2 = scanner.nextDouble();
        double y2 = scanner.nextDouble();

        double dist = distance(x1, y1, x2, y2);
        System.out.printf("Distance = %.2f%n", dist);

        scanner.close();
    }

    public static double distance(double x1, double y1, double x2, double y2) {
        return Math.sqrt(Math.pow(x2 - x1, 2) + Math.pow(y2 - y1, 2));
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. ประกาศ Method: `public static double distance(double x1, double y1, double x2, double y2)`
2. คำนวณผลต่างพิกัด: $\Delta x = x_2 - x_1$ และ $\Delta y = y_2 - y_1$
3. ใช้สูตรระยะทาง: $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$
4. เขียนคำนวณใน Java: `return Math.sqrt(Math.pow(x2 - x1, 2) + Math.pow(y2 - y1, 2));`
5. ใน `main()`: รับพิกัดทั้ง 4 ค่าจาก Scanner แล้วส่งเข้า Method แสดงผลด้วยทศนิยม 2 ตำแหน่ง

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

```
ตัวอย่างการคำนวณพิกัด (x1=1.5, y1=-3.4) ถึง (x2=4.0, y2=5.0):
Δx = 4.0 - 1.5 = 2.5   -> (2.5)^2 = 6.25
Δy = 5.0 - (-3.4) = 8.4 -> (8.4)^2 = 70.56
ผลรวม = 6.25 + 70.56 = 76.81
ระยะทาง = sqrt(76.81) = 8.764...

หน้าจอ Output:
The distance between the two points is 8.76
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **คำนวณกำลังสองโดยไม่ใช้ Math.pow:** `double dx = x2 - x1; double dy = y2 - y1; return Math.sqrt(dx * dx + dy * dy);` ซึ่งทำงานเร็วกว่า
- **ระยะทางใน 3 มิติ (3D Distance):** รับ z_1, z_2 เพิ่ม แล้วคำนวณ $\sqrt{\Delta x^2 + \Delta y^2 + \Delta z^2}$
- **Manhattan Distance:** $|x_2 - x_1| + |y_2 - y_1|$ โดยใช้ `Math.abs()`

⚠️ จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)

ใช้เครื่องหมาย `^` คิดว่าเป็นการยกกำลัง เช่น `(x2 - x1)^2` ในภาษา Java ไม่สามารถใช้เครื่องหมายนี้ยกกำลังเลขทศนิยมได้ ต้องใช้ `Math.pow()` หรือคูณตัวมันเอง

ข้อที่ 19: Ex 6: การตรวจสอบความถูกต้องและจำแนกชนิดของสามเหลี่ยม (Triangle Classifier)

Geometry Logic & Method Returning String | `Lab10A_Methods1/Triangle.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

สร้างฟังก์ชันตรวจสอบความถูกต้องของด้านสามเหลี่ยม (Triangle Inequality Theorem) และจำแนกประเภทเป็น สามเหลี่ยมด้านเท่า, หน้าจั่ว, หรือด้านไม่เท่า

ซอร์สโค้ดฉบับสมบูรณ์ (Triangle.java):

```
import java.util.Scanner;

public class Triangle {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter sides: ");
        double a = sc.nextDouble(), b = sc.nextDouble(), c = sc.nextDouble();
        System.out.println(classifyTriangle(a, b, c));
        sc.close();
    }

    public static String classifyTriangle(double a, double b, double c) {
        if (a + b <= c || a + c <= b || b + c <= a)
            return "These sides cannot form a valid triangle.";
        if (a == b && b == c)
            return "The sides form an Equilateral triangle.";
        if (a == b || b == c || a == c)
            return "The sides form an Isosceles triangle.";
        return "The sides form a Scalene triangle.";
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. ประกาศ Method: `public static String classifyTriangle(double a, double b, double c)`
2. **ขั้นตอนที่ 1: ตรวจสอบความถูกต้อง (Validity Check):**
   ผลรวมของความยาวด้านสองด้านใดๆ ต้องมากกว่าด้านที่สามเสมอ:
   `if (a + b <= c || a + c <= b || b + c <= a) return "Invalid: Not a triangle.";`
3. **ขั้นตอนที่ 2: จำแนกสามเหลี่ยมด้านเท่า (Equilateral):**
   ด้านทั้งสามยาวเท่ากันทั้งหมด:
   `if (a == b && b == c) return "The sides form an Equilateral triangle.";`
4. **ขั้นตอนที่ 3: จำแนกสามเหลี่ยมหน้าจั่ว (Isosceles):**
   มีสองด้านใดๆ ยาวเท่ากัน:
   `else if (a == b || b == c || a == c) return "The sides form an Isosceles triangle.";`
5. **ขั้นตอนที่ 4: สามเหลี่ยมด้านไม่เท่า (Scalene):**
   `else return "The sides form a Scalene triangle.";`

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

ทดสอบกรณีต่างๆ:
1. ด้าน (3, 3, 3) -> a==b && b==c -> "Equilateral triangle"
2. ด้าน (5, 5, 8) -> a==b -> "Isosceles triangle"

```
3. ด้าน (3, 4, 5) -> ไม่เท่ากันสักด้าน -> "Scalene triangle"
4. ด้าน (1, 2, 5) -> 1 + 2 <= 5 -> "Invalid: Not a triangle."
```

🔄 **แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)**

- **แยกเป็น Method boolean ตรวจสอบความถูกต้อง:** `public static boolean isValidTriangle(double a, double b, double c)`
- **คำนวณพื้นที่สามเหลี่ยมด้วยสูตรเฮรอน (Heron's Formula):** $s = (a+b+c)/2$ และ $\text{Area} = \sqrt{s(s-a)(s-b)(s-c)}$
- **เช็คสามเหลี่ยมมุมฉาก (Right Triangle):** ใช้ทฤษฎีบทพีทาโกรัส $a^2 + b^2 == c^2$

⚠️ **จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)**

ลำดับเงื่อนไขผิด: ถ้าเช็ค Isosceles (สองด้านเท่ากัน) *ก่อน* เช็ค Equilateral (สามด้านเท่ากัน) สามเหลี่ยมด้านเท่าจะถูกดักจับกลายเป็นหน้าจั่วทันที เพราะสามเหลี่ยมด้านเท่าก็มีสองด้านเท่ากันเข้าเงื่อนไข isosceles เสมอ ต้องเช็ค **Equilateral ก่อนเสมอ**

บทที่ 4: Methods Part 2: Void, Modular Design & Pass-by-Value (Lab 10B — 6 Exercises)

ภาพรวมเนื้อหา: กลไก Pass-by-Value, Helper Methods คืนค่า boolean, Void Methods วาดกรอบข้อความ, และตารางสูตรคูณ

ข้อที่ 20: Ex 1: การส่งผ่านค่าแบบ Pass-by-Value ในภาษา Java

Parameter Passing Mechanics `Lab10B_NestedMethods_VoidMethods/PassByValue.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

พิสูจน์และทำความเข้าใจกลไก Pass-by-Value ของตัวแปรชนิด Primitive (int, double, boolean) ว่าการแก้ไขค่าพารามิเตอร์ภายใน Method จะไม่ส่งผลกระทบต่อตัวแปรต้นทางในฟังก์ชันผู้เรียก

ซอร์สโค้ดฉบับสมบูรณ์ (PassByValue.java):

```
public class PassByValue {
    public static void main(String[] args) {
        int num = 10;
        System.out.println("Before: num = " + num);
        increment(num);
        System.out.println("After:  num = " + num);
    }

    public static void increment(int n) {
        n = n + 1;
        System.out.println("Inside: n = " + n);
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. ใน `main()` ประกาศตัวแปร `int x = 1;`
2. แสดงค่าก่อนเรียก Method: `"Before the call, x is 1"`
3. เรียกใช้ Method: `increment(x);` โดย Java จะทำการ **Copy ค่า 1** ไปใส่ไว้ในพารามิเตอร์ n ของ Method
4. ใน Method `increment(int n)`: สั่ง `n++;` ทำให้ตัวแปร copy (n) มีค่ากลายเป็น 2
5. กลับมาที่ `main()`: พิมพ์ค่าตัวแปร x อีกครั้ง ค่าของ x ใน main ยังคงเป็น **1** เท่าเดิมไม่เปลี่ยนแปลง

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

```
ลำดับและสถานะตัวแปรในหน่วยความจำ:
1. main() เริ่มทำงาน: สร้างกล่องตัวแปร x = 1 ใน main stack frame
2. เรียก increment(x): สร้าง stack frame ใหม่สำหรับ increment มีกล่อง n = 1 (copy จาก x)
3. ภายใน increment: n++ -> กล่อง n เปลี่ยนเป็น 2
4. increment ทำงานจบ: stack frame ของ increment ถูกทำลายทิ้งไป
5. main() ทำงานต่อ: กล่องตัวแปร x ยังคงมีค่าเป็น 1 เสมอ!

หน้าจอ Output:
Before the call, x is 1
n inside the method is 2
After the call, x is 1
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **ต้องการให้ค่าใน main เปลี่ยน:** ต้องให้ Method คืนค่ากลับมา แล้วรับค่าทับ: `x = increment(x);` โดย Method นิยาม `public static int increment(int n) { return n + 1; }`
- **เปรียบเทียบกับ Array (Pass-by-Reference):** หากส่ง Array เข้าไป การแก้ค่าสมาชิกใน Array ภายใน Method *จะทำให้ Array ต้นทางเปลี่ยนไปด้วยจริง*

⚠️ **จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)**

เข้าใจผิดว่าเมื่อตั้งชื่อพารามิเตอร์ให้ตรงกับชื่อตัวแปรใน main (เช่น ใช้ `int x` เหมือนกันทั้งใน main และใน method) แล้วค่าจะเปลี่ยนตามกัน ใน Java เป็นคนละตัวแปรและอยู่คนละขอบเขต (Scope) เสมอ

ข้อที่ 21: Ex 2: การแยกฟังก์ชันหา ห.ร.ม. แบบ Modular Design

Modular Algorithm Design `Lab10B_NestedMethods_VoidMethods/GCDModular.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

ฝึกการแยกส่วนตรรกะการคำนวณ ห.ร.ม. ออกจากฟังก์ชัน main เพื่อให้โค้ดเป็นระเบียบ อ่านง่าย และสามารถนำฟังก์ชัน `gcd(n1, n2)` ไปเรียกซ้ำในโจทย์อื่นๆ ได้

ซอร์สโค้ดฉบับสมบูรณ์ (GCDModular.java):

```
import java.util.Scanner;

public class GCDModular {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter n1: ");
        int n1 = input.nextInt();
        System.out.print("Enter n2: ");
        int n2 = input.nextInt();

        int result1 = gcd(n1, n2);
        System.out.println("GCD(" + n1 + ", " + n2 + ") = " + result1);

        int newN1 = n1 * 2;
        int newN2 = n2 * 3;
        int result2 = gcd(newN1, newN2);
        System.out.println("GCD(" + newN1 + ", " + newN2 + ") = " + result2);
    }

    public static int gcd(int n1, int n2) {
        int gcd = 1;
        int min = Math.min(n1, n2);
        for (int k = 2; k <= min; k++) {
            if (n1 % k == 0 && n2 % k == 0) {
                gcd = k;
            }
        }
        return gcd;
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. แยกการคำนวณออกเป็น Method แยกต่างหาก:
   `public static int gcd(int n1, int n2)`
2. ภายใน Method: ทำงานด้วย Brute-force หรือ Euclidean Algorithm แล้วส่งค่า `return gcd;`
3. ในฟังก์ชัน `main()`: รับหน้าที่เฉพาะการติดต่อกับผู้ใช้ (User Interface) รับตัวเลข n1, n2 จาก Scanner
4. เรียกใช้ฟังก์ชัน `int result = gcd(n1, n2);` แล้วนำผลลัพธ์มาพิมพ์แสดงอย่างสวยงาม

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

```
การส่งผ่านข้อมูล:
main() รับค่า: n1 = 45, n2 = 75
main() ส่งค่าไปยัง: gcd(45, 75)
```

```
gcd() คำนวณ: ตัวหารร่วมมากคือ 15 แล้ว return 15
main() รับผลลัพธ์และพิมพ์: "The greatest common divisor for 45 and 75 is 15"
```

🔁 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **รองรับตัวเลขติดลบ:** ใส่ `n1 = Math.abs(n1); n2 = Math.abs(n2);` ที่บรรทัดแรกของ gcd method
- **หา ห.ร.ม. ของ 3 ตัวเลข:** เรียกซ้อนกัน `int result = gcd(gcd(a, b), c);`
- **ตรวจสอบว่าเป็นจำนวนเฉพาะสัมพัทธ์ (Coprime) หรือไม่:** `boolean isCoprime = (gcd(a, b) == 1);`

⚠️ จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)

เขียนคำสั่งพิมพ์ System.out.println() ไว้ภายใน Method gcd() แทนที่จะ return ค่าออกมา การทำเช่นนี้ทำให้ Method ไม่สามารถนำผลลัพธ์ไปคำนวณต่อในจุดอื่นได้ Method คำนวณที่ดีควร **return ค่า** ออกมา

ข้อที่ 22: Ex 3: สถาปัตยกรรมแยกส่วนฟังก์ชันจำนวนเฉพาะ (Helper Method & Display Method)

Boolean Helper Method & Separation of Concerns `Lab10B_NestedMethods_VoidMethods/PrimeMethod.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

เรียนรู้การแบ่งหน้าที่การทำงาน (Separation of Concerns) โดยสร้าง Helper Method ชื่อ `isPrime()` คืนค่า boolean เพื่อนำไปใช้ร่วมกับ Method จัดการแสดงผล

ซอร์สโค้ดฉบับสมบูรณ์ (PrimeMethod.java):

```
import java.util.Scanner;

public class PrimeMethod {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter max: ");
        int max = input.nextInt();
        findPrimes(max);
    }

    public static boolean isPrime(int n) {
        if (n <= 1)
            return false;
        // เช็คการหารลงตัวแค่ถึงรากที่สองของ n ก็พอ (เพื่อความรวดเร็ว)
        for (int i = 2; i <= Math.sqrt(n); i++) {
            if (n % i == 0)
                return false;
        }
        return true;
    }

    public static void findPrimes(int max) {
        int count = 0;
        for (int i = 2; i <= max; i++) {
            if (isPrime(i)) {
                System.out.print(i + " ");
                count++;
                // พิมพ์ขึ้นบรรทัดใหม่เมื่อครบ 10 ตัว
                if (count % 10 == 0) {
                    System.out.println();
                }
            }
        }
        // ถ้าตัวสุดท้ายไม่ได้จบที่ตัวที่ 10 พอดี ให้ขึ้นบรรทัดใหม่ก่อนสรุปผล
        if (count % 10 != 0 && count > 0) {
            System.out.println();
        }
        System.out.println("Total primes found: " + count);
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. สร้าง Helper Method: `public static boolean isPrime(int number)`
   - หากตัวเลข <= 1 คืนค่า false ทันที
   - วนลูปทดสอบตัวหาร 2 ถึง number / 2: หากหารลงตัว `return false;` ทันที (Early Exit)
   - หากผ่านลูปได้โดยไม่มีตัวหารลงตัว แสดงว่าเป็นจำนวนเฉพาะ `return true;`

2. สร้าง Display Method: `public static void printPrimeNumbers(int numberOfPrimes)`
   - วนลูปค้นหาและนับจำนวนเฉพาะโดยเรียกใช้ `if (isPrime(number))`
   - จัดรูปแบบการพิมพ์ 10 จำนวนต่อบรรทัด
3. ใน `main()`: เรียกเพียงคำสั่งเดียว `printPrimeNumbers(50);`

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

```
การเรียกทำงานร่วมกัน:
printPrimeNumbers(50) เริ่มที่ number = 2
-> เรียก isPrime(2) -> return true -> พิมพ์ 2, count = 1
-> เรียก isPrime(3) -> return true -> พิมพ์ 3, count = 2
-> เรียก isPrime(4) -> return false -> ไม่พิมพ์
...
จนกระทั่ง count = 50 จึงหยุดการทำงาน
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **นับจำนวนเฉพาะในช่วง [start, end]:** สร้าง Method `public static int countPrimes(int start, int end)`
- **คืนค่าเป็น Array ของจำนวนเฉพาะ:** `public static int[] getFirstNPrimes(int n)`
- **Optimized Primality:** ตรวจเฉพาะเลขคี่หลังเช็คเลข 2 แล้ว

⚠️ จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)

ลืมดักจับกรณีเลข 0 หรือ 1 ในฟังก์ชัน `isPrime()` เลข 1 ไม่ใช่จำนวนเฉพาะตามนิยามคณิตศาสตร์ หากไม่ดัก `if (number <= 1) return false;` จะทำให้โปรแกรมมอง 1 เป็นจำนวนเฉพาะ

ข้อที่ 23: Ex 4: การค้นหาจำนวนสมบูรณ์ (Perfect Numbers up to 10,000)

Divisor Summation & Number Theory `Lab10B_NestedMethods_VoidMethods/Perfectnumber.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

เข้าใจนิยามของจำนวนสมบูรณ์ (Perfect Number คือ จำนวนเต็มบวกที่มีค่าเท่ากับผลรวมของตัวหารแท้ทั้งหมดที่ไม่รวมตัวมันเอง เช่น 6 = 1 + 2 + 3) และการค้นหาในช่วงตัวเลขขนาดใหญ่

ซอร์สโค้ดฉบับสมบูรณ์ (Perfectnumber.java):

```
public class Perfectnumber {
    public static void main(String[] args) {
        // วนลูปเพื่อหา Perfect Number ที่มีค่าน้อยกว่า 10000
        for (int i = 1; i < 10000; i++) {
            if (isPerfect(i)) {
                System.out.println(i + " is a perfect number");
            }
        }
    }

    public static boolean isPerfect(int number) {
        int sum = 0;
        // ลูปหาตัวหารทั้งหมด (ยกเว้นตัวมันเอง)
        for (int k = 1; k < number; k++) {
            // ถ้าหารลงตัว ให้นำไปบวกใน sum
            if (number % k == 0) {
                sum += k;
            }
        }
        // ถ้าผลรวมเท่ากับตัวมันเองพอดี จะถือว่าเป็น Perfect Number (return true)
        return sum == number;
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. สร้าง Method ตรวจสอบ: `public static boolean isPerfect(int n)`
2. ภายใน Method: สร้างตัวแปรสะสมผลรวมตัวหาร `int sum = 0;`
3. วนลูปหาตัวหารแท้: `for (int d = 1; d < n; d++)` (ย้ำว่า d < n ไม่รวมตัวมันเอง)
4. ถ้า d หาร n ลงตัว: `if (n % d == 0) sum += d;`
5. ส่งผลลัพธ์: `return (sum == n);` (จริงเมื่อผลรวมตัวหารเท่ากับ n พอดี)
6. ใน `main()`: วนลูปทดสอบตั้งแต่ 1 ถึง 10,000 หาก `isPerfect(i)` เป็นจริงให้แสดงผล

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

```
การตรวจสอบตัวเลขตัวอย่าง:
- เลข 6: ตัวหารแท้คือ 1, 2, 3 -> sum = 1 + 2 + 3 = 6 == 6 -> True (เป็น Perfect Number!)
- เลข 28: ตัวหารแท้คือ 1, 2, 4, 7, 14 -> sum = 28 == 28 -> True (เป็น Perfect Number!)
- เลข 10: ตัวหารแท้คือ 1, 2, 5 -> sum = 8 != 10 -> False

ผลการรันหาตั้งแต่ 1 ถึง 10,000:
The perfect numbers between 1 and 10000 are:
6
28
```

```
496
8128
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **เพิ่มประสิทธิภาพ:** วนลูปหาตัวหารแค่ `d <= n / 2` เพราะตัวหารแท้ที่ใหญ่ที่สุดไม่มีทางเกินครึ่งหนึ่งของตัวเลข
- **จำนวนบกพร่อง (Deficient) และจำนวนเกิน (Abundant):** ถ้า sum < n เรียก Deficient, ถ้า sum > n เรียก Abundant
- **พิมพ์ตัวหารแท้แจกแจงออกมา:** แสดง `6 = 1 + 2 + 3`

⚠️ จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)

วนลูปโดยใช้ `d <= n` (มีเครื่องหมายเท่ากับ) ทำให้ตัวเลขตัวมันเองถูกบวกเข้าไปใน sum ด้วย ผลรวมจะกลายเป็น $2n$ เสมอ และไม่มีทางเท่ากับ n ส่งผลให้หาจำนวนสมบูรณ์ไม่เจอเลยแม้แต่ตัวเดียว

ข้อที่ 24: Ex 5: การสร้างกรอบข้อความด้วย Void Methods เรียกซ้อนกัน

Void Methods & Character Composition `Lab10B_NestedMethods_VoidMethods/TextBox.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

สร้าง Void Methods ที่ไม่มีการคืนค่า เพื่อทำงานด้านการพิมพ์ข้อความ จัดองค์ประกอบทางสายตา (Visual Box Layout) และเรียกใช้ Void Method ย่อยต่อกันเป็นทอดๆ

ซอร์สโค้ดฉบับสมบูรณ์ (TextBox.java):

```
import java.util.Scanner;
public class TextBox {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter message: ");
        String message = input.nextLine();
        System.out.print("Enter box width: ");
        int width = input.nextInt();

        printTextBox(message, width);
    }

    public static void printLine(int width, char ch) {
        for (int i = 0; i < width; i++) {
            System.out.print(ch);
        }
        System.out.println();
    }

    public static void printCentered(String message, int width) {
        int padding = (width - message.length() - 2) / 2;
        System.out.print("*");
        for (int i = 0; i < padding; i++) {
            System.out.print(" ");
        }
        System.out.print(message);

        // เผื่อกรณีที่ความกว้างและข้อความหารกันแล้วเหลือเศษ
        // ต้องเติมช่องว่างด้านหลังให้พอดี
        int rightPadding = width - message.length() - 2 - padding;
        for (int i = 0; i < rightPadding; i++) {
            System.out.print(" ");
        }
        System.out.println("*");
    }

    public static void printTextBox(String message, int width) {
        printLine(width, '*');
        printCentered(message, width);
        printLine(width, '*');
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. Method พิมพ์เส้นขอบแนวนอน: `public static void printBorder(char ch, int width)`
   วนลูปพิมพ์ตัวอักษร ch จำนวน width ตัว แล้วขึ้นบรรทัดใหม่

2. Method พิมพ์ข้อความตรงกลาง: `public static void printCentered(String text, int width)`
   คำนวณช่องว่างซ้าย-ขวา: `(width - text.length() - 2) / 2` แล้วพิมพ์ขอบซ้าย ช่องว่าง ข้อความ และขอบขวา
3. Method ประกอบกล่อง: `public static void printBox(String text, int width)`
   เรียก `printBorder('+', width);` (ขอบบน)
   เรียก `printCentered(text, width);` (ข้อความกลาง)
   เรียก `printBorder('+', width);` (ขอบล่าง)

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

```
ตัวอย่างผลลัพธ์เมื่อเรียก printBox("Hello Java", 24):
++++++++++++++++++++++++
+      Hello Java      +
++++++++++++++++++++++++

ความกว้างทั้งหมด: 24 ตัวอักษร
ข้อความ: 10 ตัวอักษร, ขอบ: 2 ตัวอักษร
ช่องว่างซ้าย-ขวา: (24 - 10 - 2) / 2 = 6 ช่องเท่ากันสองฝั่ง
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **กล่องหลายบรรทัด:** รับเป็น Array ของ String หรือรับ String ที่มีเครื่องหมาย \n
- **กล่องแบบมีเส้นขอบข้างว่างเปล่า:** เพิ่มความสูงของกล่องโดยใส่บรรทัดว่าง `printEmptyLine(width);` ด้านบนและล่างข้อความ
- **เปลี่ยนสไตล์กรอบ:** รับตัวอักษรมุม '+' และตัวอักษรขอบ '-' หรือ '|'

⚠️ จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)

พยายามนำผลลัพธ์ของ Void Method ไปกำหนดค่าให้กับตัวแปร เช่น `int result = printBorder('*', 10);` จะเกิดข้อผิดพลาด `Type mismatch: cannot convert from void to int` เพราะ void method ไม่มีการคืนค่าใดๆ ทั้งสิ้น

ข้อที่ 25: Ex 6: การพิมพ์ตารางสูตรคูณแม่ 1-9 ด้วยระบบ Void Methods

Structured Table & Coordinated Void Methods | `Lab10B_NestedMethods_VoidMethods/MultiTable.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

ฝึกการออกแบบโปรแกรมพิมพ์ตารางขนาดใหญ่ที่ซับซ้อน โดยแบ่งความรับผิดชอบออกเป็น Method ย่อยๆ: พิมพ์หัวตาราง, พิมพ์เส้นแบ่ง, และพิมพ์แถวข้อมูล

ซอร์สโค้ดฉบับสมบูรณ์ (MultiTable.java):

```
import java.util.Scanner;

public class MultiTable {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter table size: ");
        int size = sc.nextInt();
        printMultiTable(size);
        sc.close();
    }

    public static void printDivider(int cols) {
        System.out.print("+----");
        for (int i = 0; i < cols; i++)
            System.out.print("+----");
        System.out.println("+");
    }

    public static void printRow(int row, int cols) {
        System.out.printf("%4d |", row);
        for (int j = 1; j <= cols; j++)
            System.out.printf("%5d", row * j);
        System.out.println();
    }

    public static void printMultiTable(int size) {
        System.out.printf("%5s|", " ");
        for (int j = 1; j <= size; j++)
            System.out.printf("%5d", j);
        System.out.println();
        printDivider(size);
        for (int i = 1; i <= size; i++) {
            printRow(i, size);
            printDivider(size);
        }
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. Method พิมพ์หัวคอลัมน์: `public static void printHeader()` พิมพ์หมายเลข 1 ถึง 9 ในระยะห่างคอลัมน์ที่เท่ากัน
2. Method พิมพ์เส้นคั่นตาราง: `public static void printDivider()` พิมพ์เส้นประและเครื่องหมายบวก เช่น `"----+----------"`
3. Method พิมพ์เนื้อหาตาราง: `public static void printTable()` วนลูปแถว `row = 1..9` และลูปคอลัมน์ `col = 1..9`
   - พิมพ์หมายเลขแถวและเครื่องหมาย `" | "`
   - พิมพ์ผลคูณ `row * col` ด้วย `printf("%4d", row * col)`

4. ใน `main()`: เรียกต่อกันตามลำดับ:

```
printHeader();
printDivider();
printTable();
```

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

หน้าตาตารางสูตรคูณที่ได้:

```
       1   2   3   4   5   6   7   8   9
---+------------------------------------
 1 |   1   2   3   4   5   6   7   8   9
 2 |   2   4   6   8  10  12  14  16  18
 3 |   3   6   9  12  15  18  21  24  27
 4 |   4   8  12  16  20  24  28  32  36
 ...
 9 |   9  18  27  36  45  54  63  72  81
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **ตารางขนาด N x N ตามสั่ง:** ให้รับค่าขนาดสูงสุด `maxN` จากผู้ใช้แล้วสร้างตารางตามขนาดนั้น
- **ตารางเฉพาะสามเหลี่ยมล่าง:** วนลูปพิมพ์เฉพาะเมื่อ `col <= row`
- **ตารางการบวก (Addition Table):** เปลี่ยนจากผลคูณ `row * col` เป็นผลบวก `row + col`

⚠️ จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)

ลืมพิมพ์คำสั่ง `System.out.println();` เพื่อขึ้นบรรทัดใหม่หลังจบลูปคอลัมน์ในแต่ละแถว ทำให้ตัวเลขผลคูณทั้งหมดของทุกแถวไหลต่อกันเป็นเส้นยาวในบรรทัดเดียว

บทที่ 5: 1D Arrays (Lab 11A — 6 Exercises)

ภาพรวมเนื้อหา: โครงสร้าง Indexing, การรับค่าแบบ Dynamic, สถิติ Sum/Avg/Min/Max, Linear Search, กรองข้อมูล, และ In-place Swap

ข้อที่ 26: Ex 1: โครงสร้างหน่วยความจำและดัชนีของ 1D Array (Array Indexing & Traversal)

Array Basics & Indexing | `Lab11A_1DArrays/TraceArray.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

เข้าใจการจองพื้นที่หน่วยความจำของ Array ใน Java, ค่าเริ่มต้น (Default Value), การอ้างอิงผ่าน Index เริ่มจาก 0 ถึง length - 1, และการใช้ Attribute .length

ซอร์สโค้ดฉบับสมบูรณ์ (TraceArray.java):

```
public class TraceArray {
    public static void main(String[] args) {
        int[] nums = {5, 12, 3, 8, 7};
        int total = 0;
        for (int i = 0; i < nums.length; i++) {
            total = total + nums[i];
            System.out.println("i=" + i + "  nums[i]=" + nums[i] + "  total=" + total);
        }
        System.out.println("Final total = " + total);
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. สร้าง Array ขนาด 5 ช่อง: `int[] arr = new int[5];` (ทุกช่องมีค่าเริ่มต้นอัตโนมัติเป็น 0)
2. กำหนดค่าให้แต่ละช่องด้วยลูป: `for (int i = 0; i < arr.length; i++) arr[i] = (i + 1) * 10;`
3. วนลูปอ่านค่าและพิมพ์ออกทางหน้าจอ: `for (int i = 0; i < arr.length; i++) System.out.println("arr[" + i + "] = " + arr[i]);`
4. ตรวจสอบขนาดของ Array: ใช้ `arr.length` (ไม่มีวงเล็บ)

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

```
โครงสร้างในหน่วยความจำ Heap:
Index:   [ 0 ]   [ 1 ]   [ 2 ]   [ 3 ]   [ 4 ]
Value:     10       20       30       40       50

ความยาว (arr.length): 5
ดัชนีตัวแรก: arr[0] = 10
ดัชนีตัวสุดท้าย: arr[arr.length - 1] = arr[4] = 50

หน้าจอ Output:
arr[0] = 10
arr[1] = 20
arr[2] = 30
arr[3] = 40
arr[4] = 50
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **ประกาศพร้อมกำหนดค่าทันที (Array Initializer):** `int[] arr = {10, 20, 30, 40, 50};`
- **ใช้ For-each loop อ่านค่า:** `for (int val : arr) System.out.print(val + " ");` (เหมาะกับการอ่าน ไม่เหมาะกับการแก้ไขค่า)
- **Array ชนิดอื่น:** `double[]` (default 0.0), `boolean[]` (default false), `String[]` (default null)

⚠️ **จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)**

ArrayIndexOutOfBoundsException: พยายามเข้าถึง Index ที่เท่ากับหรือเกินขนาด เช่น เขียน `for (int i = 0; i <= arr.length; i++)` เมื่อ i เท่ากับ 5 จะเกิดข้อผิดพลาดทำให้โปรแกรมหยุดทำงานทันที

ข้อที่ 27: Ex 2: การจองขนาด Array แบบไดนามิกตาม Input ผู้ใช้ (Dynamic Array Input)

User Input & Dynamic Allocation `Lab11A_1DArrays/ArrayInput.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

เรียนรู้การรับขนาด Array จากผู้ใช้ในขณะที่โปรแกรมทำงาน (Runtime), การสร้าง Array ด้วยขนาดที่กำหนด, และการวนลูปรับข้อมูลลง Array ทีละช่อง

ซอร์สโค้ดฉบับสมบูรณ์ (ArrayInput.java):

```
import java.util.Scanner;

public class ArrayInput {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter array size (3-10): ");
        int n = sc.nextInt();

        int[] arr = new int[n]; // สร้างอาร์เรย์ขนาด n
        for (int i = 0; i < n; i++) {
            System.out.print("Enter element [" + i + "]: ");
            arr[i] = sc.nextInt();
        }

        System.out.println("Array contents:");
        for (int i = 0; i < arr.length; i++) {
            System.out.println("arr[" + i + "] = " + arr[i]);
        }
        sc.close();
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. รับจำนวนสมาชิกทั้งหมดจากคีย์บอร์ด: `int n = sc.nextInt();`
2. สร้าง Array ขนาด n ช่อง: `int[] numbers = new int[n];`
3. วนลูปรับค่าสมาชิกทีละตัว:

```
for (int i = 0; i < numbers.length; i++) {
    System.out.print("Enter element " + (i + 1) + ": ");
    numbers[i] = sc.nextInt();
}
```

4. วนลูปแสดงข้อมูลที่ได้รับทั้งหมดเพื่อยืนยันความถูกต้อง

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

ตัวอย่างการรัน:

```
Enter the number of elements: 4
Enter element 1: 15
Enter element 2: 42
Enter element 3: 8
Enter element 4: 99

You entered: [15, 42, 8, 99]
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **รับข้อมูลในบรรทัดเดียว:** ผู้ใช้พิมพ์ตัวเลขคั่นด้วยช่องว่าง `15 42 8 99` แล้วกด Enter ครั้งเดียว Scanner จะดึงข้อมูลทีละตัวได้อย่างถูกต้อง
- **รับ Array ของทศนิยม (double[]):** เปลี่ยนชนิดตัวแปรและใช้ `sc.nextDouble()`
- **ดักจับขนาดไม่ถูกต้อง:** ตรวจสอบ `if (n <= 0)` ก่อนสร้าง Array ป้องกัน `NegativeArraySizeException`

⚠️ จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)

สับสนระหว่าง Index ที่แสดงให้ผู้ใช้เห็นกับ Index จริงใน Array ผู้ใช้ทั่วไปคุ้นชินกับการเริ่มนับชิ้นที่ 1 (Element 1) แต่ในโค้ดต้องเก็บที่ `numbers[0]` เสมอ

ข้อที่ 28: Ex 3: การคำนวณสถิติครบวงจรในรอบเดียว (Sum, Average, Min, Max in Single Pass)

Array Statistics & Reduction | `Lab11A_1DArrays/ArrayStats.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

ฝึกการเขียนอัลกอริทึมคำนวณสถิติพื้นฐาน 4 ค่า (ผลรวม, ค่าเฉลี่ย, ค่าน้อยสุด, และค่ามากสุด) พร้อมกันในลูปเดียวอย่างมีประสิทธิภาพ

ซอร์สโค้ดฉบับสมบูรณ์ (ArrayStats.java):

```
import java.util.Scanner;

public class ArrayStats {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter array size (3-10): ");
        int n = sc.nextInt();

        int[] arr = new int[n];
        int sum = 0;

        // รับค่าและคำนวณ sum ไปพร้อมกัน
        for (int i = 0; i < n; i++) {
            System.out.print("Enter element [" + i + "]: ");
            arr[i] = sc.nextInt();
            sum += arr[i]; // สะสมค่า sum ทันทีที่รับค่าเข้ามา
        }

        // หาค่า Minimum และ Maximum
        int min = arr[0];
        int max = arr[0];
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] < min) {
                min = arr[i];
            }
            if (arr[i] > max) {
                max = arr[i];
            }
        }

        double average = (double) sum / arr.length;

        // แสดงผลลัพธ์
        System.out.println("Sum     = " + sum);
        System.out.printf("Average = %.2f\n", average);
        System.out.println("Minimum = " + min);
        System.out.println("Maximum = " + max);

        sc.close();
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. รับข้อมูลเข้า Array ขนาด n
2. **กำหนดค่าเริ่มต้นสถิติ:**
   - `int sum = arr[0];` (หรือ 0)
   - `int min = arr[0];` (ต้องใช้สมาชิกตัวแรกเสมอ!)
   - `int max = arr[0];` (ต้องใช้สมาชิกตัวแรกเสมอ!)

3. วนลูปตรวจสอบตั้งแต่สมาชิกตัวที่ 2 เป็นต้นไป (Index 1 ถึง n - 1):

```
for (int i = 1; i < arr.length; i++) {
    sum += arr[i];
    if (arr[i] < min) min = arr[i];
    if (arr[i] > max) max = arr[i];
}
```

4. คำนวณค่าเฉลี่ย: `double avg = (double) sum / arr.length;` (ห้ามลืม cast เป็น double)
5. แสดงผลลัพธ์ทั้ง 4 ค่าด้วย printf

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

```
ข้อมูลตัวอย่าง: arr = [18, 5, 42, 7, 23]
เริ่มต้น: min = 18, max = 18, sum = 18
i = 1 (ค่า 5):  sum = 23, min = 5,  max = 18
i = 2 (ค่า 42): sum = 65, min = 5,  max = 42
i = 3 (ค่า 7):  sum = 72, min = 5,  max = 42
i = 4 (ค่า 23): sum = 95, min = 5,  max = 42

ผลลัพธ์:
Sum     = 95
Average = 19.00
Minimum = 5
Maximum = 42
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **แยกเป็น Method ย่อย 4 ฟังก์ชัน:** `getSum(arr)`, `getAverage(arr)`, `getMin(arr)`, `getMax(arr)`
- **หาตำแหน่ง Index ของ Min และ Max (ArgMin / ArgMax):** เก็บ `minIdx = i` และ `maxIdx = i` แทนที่จะเก็บเฉพาะตัวเลข
- **คำนวณส่วนเบี่ยงเบนมาตรฐาน (Standard Deviation):** วนลูปอีกรอบหา $\sum (x_i - \bar{x})^2$

⚠️ จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)

กับดักคะแนนหาย: ตั้งค่า `int min = 0;` ถ้าใน Array มีเฉพาะเลขบวก เช่น [10, 25, 40] ค่า min จะไม่ถูกอัปเดตเลยและตอบ 0 ออกมาเสมอ หรือถ้าตั้ง `int max = 0;` แล้วข้อมูลมีแต่เลขติดลบ [-5, -12, -2] จะตอบ max เป็น 0 ซึ่งผิดทั้งคู่! กฎเหล็กคือต้องตั้ง `min = arr[0]` และ `max = arr[0]`

ข้อที่ 29: Ex 4: การค้นหาข้อมูลตามลำดับ (Linear Search & Early Break)

Searching Algorithms `Lab11A_1DArrays/LinearSearch.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

สร้างอัลกอริทึมค้นหาค่าเป้าหมาย (Target) ใน Array แบบ Linear Search โดยเปรียบเทียบทีละตัวตั้งแต่ต้นจนจบ และหยุดค้นหาทันทีเมื่อพบ (Early Break)

ซอร์สโค้ดฉบับสมบูรณ์ (LinearSearch.java):

```
import java.util.Scanner;

public class LinearSearch {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter array size (3-10): ");
        int n = sc.nextInt();

        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            System.out.print("Enter element [" + i + "]: ");
            arr[i] = sc.nextInt();
        }

        System.out.print("Enter target to search: ");
        int target = sc.nextInt();
        int foundAt = -1; // -1 หมายถึงยังไม่พบ

        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == target) {
                foundAt = i;
                break; // หยุดลูปทันทีที่เจอ
            }
        }

        if (foundAt != -1) {
            System.out.println("Found " + target + " at index " + foundAt);
        } else {
            System.out.println(target + " not found in the array.");
        }

        sc.close();
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. มี Array ข้อมูล และรับค่าเป้าหมายที่ต้องการค้นหา `int target = sc.nextInt();`
2. สร้างตัวแปรเก็บตำแหน่งผลลัพธ์: `int foundIndex = -1;` (กำหนดค่าเริ่มต้นเป็น -1 เพื่อเป็นตัวแทนของ 'ยังไม่พบ')
3. วนลูปตรวจสอบสมาชิกทีละตัว: `for (int i = 0; i < arr.length; i++)`
4. หากพบว่า `arr[i] == target`:
   - บันทึกตำแหน่ง: `foundIndex = i;`
   - สั่ง `break;` ทันทีเพื่อไม่ให้เสียเวลาค้นหาต่อในช่องที่เหลือ
5. หลังจบลูป: ตรวจสอบ `if (foundIndex != -1)` พบข้อมูลที่ Index นั้น ถ้าเท่ากับ -1 แสดงว่าไม่พบ

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

```
ค้นหา target = 25 ใน arr = [10, 40, 25, 80, 5]
i = 0: arr[0] คือ 10 != 25 -> ค้นหาต่อ
i = 1: arr[1] คือ 40 != 25 -> ค้นหาต่อ
i = 2: arr[2] คือ 25 == 25 -> พบแล้ว! บันทึก foundIndex = 2 แล้วสั่ง break!

ผลลัพธ์: Element 25 found at index 2.
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **นับจำนวนครั้งที่พบ (Frequency Count):** หากต้องการหาว่ามีกี่ตัวใน Array ห้ามใส่ break ให้ใช้ตัวแปร `count++` เพื่อนับจนจบลูป
- **ค้นหาใน Array ของ String:** ต้องใช้ `arr[i].equals(target)` ห้ามใช้ `==`
- **ค้นหาตำแหน่งสุดท้ายที่พบ:** วนลูปจากขวามาซ้าย `for (int i = arr.length - 1; i >= 0; i--)`

⚠️ จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)

ใส่คำสั่ง `else System.out.println("Not found");` ไว้ *ภายในลูป* ส่งผลให้ทุกรอบที่ตรวจแล้วยังไม่ตรง จะพิมพ์คำว่า Not found ซ้ำออกมาเต็มหน้าจอ! การตรวจสอบว่าพบหรือไม่พบต้องทำ **หลังจบลูปเท่านั้น** โดยเช็คจากตัวแปร flag หรือ index

ข้อที่ 30: Ex 5: การกรองข้อมูลและการนับตามเงื่อนไข (Filter, Count & Above Average)

Array Filtering & Multi-pass Processing `Lab11A_1DArrays/CountFilter.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

ฝึกการประมวลผล Array แบบหลายรอบ (Multi-pass Processing) เพื่อกรองข้อมูลที่ตรงตามเกณฑ์ เช่น การหาจำนวนที่มากกว่าค่าเฉลี่ย หรือการคัดกรองเฉพาะเลขคู่

ซอร์สโค้ดฉบับสมบูรณ์ (CountFilter.java):

```
import java.util.Scanner;

public class CountFilter {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter array size (3-10): ");
        int n = sc.nextInt();

        int[] arr = new int[n];
        int sum = 0;
        for (int i = 0; i < n; i++) {
            System.out.print("Enter element [" + i + "]: ");
            arr[i] = sc.nextInt();
            sum += arr[i];
        }

        // (a) นับและแสดงเลขคู่
        System.out.print("Even numbers: ");
        int evenCount = 0;
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] % 2 == 0) {
                System.out.print(arr[i] + " ");
                evenCount++;
            }
        }
        System.out.println("  (count = " + evenCount + ")");

        // คำนวณค่าเฉลี่ย
        double avg = (double) sum / arr.length;
        System.out.printf("Average = %.2f\n", avg);

        // (b) นับและแสดงตัวเลขที่มากกว่าค่าเฉลี่ย
        System.out.print("Above average: ");
        int aboveAvgCount = 0;
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] > avg) {
                System.out.print(arr[i] + " ");
                aboveAvgCount++;
            }
        }
        System.out.println("  (count = " + aboveAvgCount + ")");

        sc.close();
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. รอบที่ 1: คำนวณผลรวมและค่าเฉลี่ยของทั้ง Array (เพราะต้องรู้ค่าเฉลี่ยก่อน จึงจะสามารถนำไปเปรียบเทียบในรอบถัดไปได้)

2. รอบที่ 2 (กรองข้อมูล): ประกาศตัวนับ `int aboveCount = 0;`
3. วนลูปตรวจสอบสมาชิกทีละตัว: `for (int i = 0; i < arr.length; i++)`
4. ตรวจสอบเงื่อนไข: `if (arr[i] > avg)`
   - พิมพ์ค่าสมาชิกตัวนั้นออกมา
   - เพิ่มตัวนับ `aboveCount++;`
5. แสดงจำนวนสมาชิกทั้งหมดที่ผ่านการกรอง

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

```
ข้อมูล: arr = [10, 25, 40, 15, 30]
รอบที่ 1: sum = 120, avg = 120 / 5.0 = 24.0
รอบที่ 2: ตรวจสอบเงื่อนไข (> 24.0):
- 10 > 24.0 (False)
- 25 > 24.0 (True)  -> เข้าเกณฑ์
- 40 > 24.0 (True)  -> เข้าเกณฑ์
- 15 > 24.0 (False)
- 30 > 24.0 (True)  -> เข้าเกณฑ์

ผลลัพธ์:
Average = 24.00
Elements above average: 25, 40, 30 (Count = 3)
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **สร้าง Array ใหม่เก็บเฉพาะค่าที่ผ่านเกณฑ์:** ต้องนับจำนวนที่ผ่านเกณฑ์ก่อน เพื่อนำไปจองขนาด Array ใหม่ `int[] filtered = new int[count];`
- **กรองตามเปอร์เซ็นต์หรือเกรด:** เช่น คัดเฉพาะคะแนนที่มากกว่า 80 คะแนน
- **กรองเลขคู่และเลขคี่แยกกัน:** ใช้เงื่อนไข `arr[i] % 2 == 0`

⚠️ จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)

พยายามกรองข้อมูลหาค่าที่มากกว่าค่าเฉลี่ยในลูปแรกที่รับข้อมูล ซึ่งเป็นไปไม่ได้ในเชิงตรรกะ เพราะยังไม่รู้ค่าเฉลี่ยของทั้งชุดข้อมูล โจทย์ลักษณะนี้จำเป็นต้องใช้ **2 ลูปเสมอ** (ลูปแรกหา avg, ลูปสองกรองข้อมูล)

ข้อที่ 31: Ex 6: การสลับตำแหน่งกลับด้าน Array แบบ In-Place (Two-Pointer In-Place Swap)

In-Place Array Manipulation `Lab11A_1DArrays/ReverseArray.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

สร้างอัลกอริทึมกลับลำดับข้อมูลใน Array จากหน้าไปหลัง โดยใช้หน่วยความจำเดิมที่มีอยู่โดยไม่สร้าง Array ใหม่ (In-Place Algorithm) ด้วยเทคนิค Two Pointers

ซอร์สโค้ดฉบับสมบูรณ์ (ReverseArray.java):

```
import java.util.Scanner;

public class ReverseArray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter array size (3-10): ");
        int n = sc.nextInt();

        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            System.out.print("Enter element [" + i + "]: ");
            arr[i] = sc.nextInt();
        }

        // พิมพ์ค่าก่อนกลับด้าน
        System.out.print("Before: ");
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }
        System.out.println();

        // อัลกอริทึม In-place Reversal (สลับซ้าย-ขวา)
        int left = 0;
        int right = arr.length - 1;
        while (left < right) {
            int temp = arr[left];
            arr[left] = arr[right];
            arr[right] = temp;
            left++;
            right--;
        }

        // พิมพ์ค่าหลังกลับด้าน
        System.out.print("After:  ");
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }
        System.out.println();

        sc.close();
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. กำหนดตัวชี้สองฝั่ง:
   - `int left = 0;` (ชี้ที่หัว Array)
   - `int right = arr.length - 1;` (ชี้ที่ท้าย Array)

2. วนลูปตราบเท่าที่ตัวชี้ทั้งสองยังไม่เดินมาชนหรือสวนทางกัน: `while (left < right)`
3. **การสลับค่า (Swap) ด้วยตัวแปรพักค่า (temp):**
    1. `int temp = arr[left];` (เก็บค่าซ้ายไว้ชั่วคราว)
    2. `arr[left] = arr[right];` (เอาค่าขวามาใส่ซ้าย)
    3. `arr[right] = temp;` (เอาค่าเดิมของซ้ายมาใส่ขวา)
4. ขยับตัวชี้: `left++;` (ขยับไปทางขวา) และ `right--;` (ขยับไปทางซ้าย)
5. เมื่อลูปจบ ข้อมูลใน Array จะถูกกลับด้านอย่างสมบูรณ์

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

```
จำลองการสลับ Array [1, 2, 3, 4, 5]:
เริ่มต้น: left = 0 (ค่า 1), right = 4 (ค่า 5)
รอบที่ 1: สลับ arr[0] กับ arr[4] -> [5, 2, 3, 4, 1]
          left เพิ่มเป็น 1, right ลดเหลือ 3
รอบที่ 2: สลับ arr[1] กับ arr[3] -> [5, 4, 3, 2, 1]
          left เพิ่มเป็น 2, right ลดเหลือ 2
รอบที่ 3: left = 2, right = 2 -> เงื่อนไข left < right เป็นเท็จ -> จบลูป!

ผลลัพธ์: Array กลับด้านเป็น [5, 4, 3, 2, 1] สำเร็จโดยใช้หน่วยความจำคงที่ O(1)
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **เขียนโดยใช้ For Loop แบบครึ่งทาง:**
  `for (int i = 0; i < arr.length / 2; i++) { int temp = arr[i]; arr[i] = arr[arr.length - 1 - i]; arr[arr.length - 1 - i] = temp; }`
- **สร้าง Array ใหม่มารับค่า (ไม่ In-place):** สร้าง `int[] rev = new int[n];` แล้วคัดลอกจากหลังมาหน้า
- **สลับเฉพาะครึ่งแรกหรือครึ่งหลัง**

⚠️ จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)

ข้อผิดพลาดร้ายแรงที่สุด: เขียนลูป `for (int i = 0; i < arr.length; i++)` โดยไม่มีเงื่อนไขหยุดที่กึ่งกลาง ลูปจะทำการสลับตัวเลขไปจนถึงตัวสุดท้าย ส่งผลให้ตัวเลขถูกสลับไปแล้วถูกสลับกลับคืนมาเป็น Array หน้าตาเหมือนเดิมก่อนสลับ!

บทที่ 6: 2D Arrays / Matrices (Lab 11B — 5 Exercises)

ภาพรวมเนื้อหา: พิกัด [row][col], การอ่าน Matrix ผ่าน Scanner, การหาผลรวมรายแถว, ค่าสูงสุดรายแถว, และ 2D Early Exit Search

ข้อที่ 32: Ex 1: โครงสร้างหน่วยความจำและระบบพิกัดของ 2D Array (Matrix Coordinate [row] [col])

2D Array Structure & Traversal | `Lab11B_2DArrays/TraceMatrix.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

เข้าใจการจัดเก็บข้อมูลแบบ 2 มิติ (Array of Arrays) ในภาษา Java, การอ่านจำนวนแถวด้วย `m.length`, การอ่านจำนวนคอลัมน์ด้วย `m[0].length`, และการเข้าถึงพิกัด `matrix[row][col]`

ซอร์สโค้ดฉบับสมบูรณ์ (TraceMatrix.java):

```
public class TraceMatrix {
    public static void main(String[] args) {
        int[][] m = { { 2, 4, 6 },
                { 1, 3, 5 },
                { 7, 0, 9 } };
        for (int row = 0; row < m.length; row++) {
            for (int col = 0; col < m[row].length; col++) {
                System.out.print(m[row][col] + " ");
            }
            System.out.println();
        }
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. สร้าง 2D Array ขนาด 3 แถว 4 คอลัมน์: `int[][] m = new int[3][4];`
2. ทำความเข้าใจขนาด:
   - `m.length` คือจำนวนแถว (Rows = 3)
   - `m[r].length` คือจำนวนคอลัมน์ของแถวที่ r (Cols = 4)
3. กำหนดค่าให้สมาชิกในพิกัด: `m[row][col] = row * 10 + col;`
4. วนลูปอ่านค่าแบบ Nested Loop:
   1. ลูปนอก (Outer): วนตามแถว `for (int r = 0; r < m.length; r++)`
   2. ลูปใน (Inner): วนตามคอลัมน์ `for (int c = 0; c < m[r].length; c++)`
   3. พิมพ์สมาชิกในแถวเดียวกันคั่นด้วยช่องว่าง
   4. ขึ้นบรรทัดใหม่หลังจบคอลัมน์ของแต่ละแถว `System.out.println();`

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

```
พิกัดและการจัดเก็บข้อมูลในตาราง (3 แถว x 4 คอลัมน์):
พิกัด:        Col 0    Col 1    Col 2    Col 3
Row 0:     m[0][0]  m[0][1]  m[0][2]  m[0][3]
Row 1:     m[1][0]  m[1][1]  m[1][2]  m[1][3]
Row 2:     m[2][0]  m[2][1]  m[2][2]  m[2][3]

หน้าจอ Output:
  0   1   2   3
 10  11  12  13
 20  21  22  23
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **สร้างพร้อมกำหนดค่าทันที (Matrix Literal):**
  `int[][] m = { {1, 2, 3}, {4, 5, 6}, {7, 8, 9} };`
- **เมทริกซ์แบบ Jagged/Ragged Array:** แต่ละแถวมีความยาวคอลัมน์ไม่เท่ากันได้ เช่น `m[0] = new int[2]; m[1] = new int[5];`
- **วนลูปแบบคอลัมน์เป็นหลัก (Column-major traversal):** สลับลูปให้ col อยู่นอก และ row อยู่ใน

⚠️ **จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)**

สลับพิกัด [col][row]: สับสนระหว่างแถวและคอลัมน์ กฎเหล็กของ Java คือวงเล็บแรกคือ **แถว (Row)** เสมอ และวงเล็บที่สองคือ **คอลัมน์ (Column)** เสมอ: `matrix[row][col]`

ข้อที่ 33: Ex 2: การรับค่าเมทริกซ์จากผู้ใช้ และการส่ง Scanner เป็น Parameter

2D Input & Resource Management `Lab11B_2DArrays/MatrixInput.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

สร้างฟังก์ชันรับข้อมูลเมทริกซ์จากผู้ใช้ที่มีขนาด $R \times C$ อย่างเป็นระบบ และฝึกการส่งต่ออ็อบเจกต์ Scanner ผ่านพารามิเตอร์เพื่อจัดการทรัพยากรอย่างถูกต้อง

ซอร์สโค้ดฉบับสมบูรณ์ (MatrixInput.java):

```
import java.util.Scanner;

public class MatrixInput {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter rows: ");
        int rows = sc.nextInt();
        System.out.print("Enter cols: ");
        int cols = sc.nextInt();
        int[][] matrix = readMatrix(sc, rows, cols);
        System.out.println("Matrix:");
        printMatrix(matrix);
        sc.close();
    }

    public static int[][] readMatrix(Scanner sc, int rows, int cols) {
        int[][] m = new int[rows][cols];
        for (int r = 0; r < rows; r++)
            for (int c = 0; c < cols; c++) {
                System.out.print("Enter element [" + r + "][" + c + "]: ");
                m[r][c] = sc.nextInt();
            }
        return m;
    }

    public static void printMatrix(int[][] m) {
        for (int r = 0; r < m.length; r++) {
            for (int c = 0; c < m[r].length; c++)
                System.out.printf("%-5d", m[r][c]);
            System.out.println();
        }
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. ใน `main()`: รับจำนวนแถวและคอลัมน์ `int rows = sc.nextInt(); int cols = sc.nextInt();`
2. ประกาศ Method รับค่า: `public static int[][] readMatrix(Scanner sc, int rows, int cols)`
3. ภายใน Method: สร้างเมทริกซ์ `int[][] matrix = new int[rows][cols];`
4. วนลูปซ้อนสองชั้นเพื่อรับค่าทีละช่อง:

```
for (int r = 0; r < rows; r++) {
    for (int c = 0; c < cols; c++) {
        System.out.print("Enter element [" + r + "][" + c + "]: ");
        matrix[r][c] = sc.nextInt();
```

```
        }
    }
```

5. ส่งเมทริกซ์ที่ได้กลับออกมา: `return matrix;`
6. สร้าง Method พิมพ์เมทริกซ์: `public static void printMatrix(int[][] matrix)`

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

```
ตัวอย่างการป้อนเมทริกซ์ขนาด 2x3:
Enter rows: 2
Enter cols: 3
Enter element [0][0]: 5
Enter element [0][1]: 8
Enter element [0][2]: 2
Enter element [1][0]: 9
Enter element [1][1]: 4
Enter element [1][2]: 1

The matrix is:
5  8  2
9  4  1
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **สร้างเมทริกซ์แบบสุ่ม (Random Matrix):** สุ่มตัวเลข 0 ถึง 99 ใส่ในแต่ละช่องแทนการรับจากคีย์บอร์ด
- **รับเมทริกซ์จัตุรัส (Square Matrix):** รับค่า n เพียงค่าเดียว แล้วสร้างขนาด $N \times N$
- **แปลงชนิดเป็น double[][]:** เพื่อรองรับข้อมูลทศนิยม

⚠️ จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)

สร้าง Scanner ซ้ำซ้อน: เขียนคำสั่ง `Scanner sc = new Scanner(System.in);` ไว้ *ภายใน Method* `readMatrix` แทนที่จะรับ Scanner จาก main การทำเช่นนี้ทำให้เกิด Object สิ้นเปลือง และถ้าเผลอสั่ง `sc.close()` ใน method จะทำให้ System.in ทั้งหมดของโปรแกรมถูกปิดตายถาวร

ข้อที่ 34: Ex 3: การคำนวณผลรวมทั้งตารางและผลรวมรายแถว (Total Sum, Row Sums & Average)

Matrix Reduction & Row Accumulation `Lab11B_2DArrays/MatrixSum.java`

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

ฝึกการคำนวณผลรวมทั้งหมดของเมทริกซ์ (Grand Total), ค่าเฉลี่ยรวมทุกช่อง, และการคำนวณผลรวมแยกเฉพาะแต่ละแถว (Row Sums)

ซอร์สโค้ดฉบับสมบูรณ์ (MatrixSum.java):

```
// SE101 - Lab 11B: Exercise 3 (MatrixSum.java)
import java.util.Scanner;

public class MatrixSum {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter rows: ");
        int rows = scanner.nextInt();
        System.out.print("Enter cols: ");
        int cols = scanner.nextInt();

        int[][] matrix = readMatrix(rows, cols);

        System.out.println("\nMatrix:");
        printMatrix(matrix);

        int total = sum(matrix);
        double avg = (double) total / (rows * cols);

        System.out.println("\nSum of all elements = " + total);
        System.out.printf("Average             = %.2f%n", avg);

        scanner.close();
    }

    public static int[][] readMatrix(int rows, int cols) {
        Scanner scanner = new Scanner(System.in);
        int[][] m = new int[rows][cols];
        for (int row = 0; row < m.length; row++) {
            for (int col = 0; col < m[row].length; col++) {
                System.out.print("Enter element [" + row + "][" + col + "]: ");
                m[row][col] = scanner.nextInt();
            }
        }
        return m;
    }

    public static void printMatrix(int[][] m) {
        for (int row = 0; row < m.length; row++) {
            for (int col = 0; col < m[row].length; col++) {
                System.out.printf("%-5d", m[row][col]);
            }
            System.out.println();
        }
    }

    public static int sum(int[][] m) {
        int total = 0;
        for (int row = 0; row < m.length; row++) {
            for (int col = 0; col < m[row].length; col++) {
```

```
                total += m[row][col];
            }
        }
        return total;
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. มีเมทริกซ์ขนาด $R \times C$
2. **คำนวณผลรวมทั้งหมด:** สร้างตัวแปร `totalSum = 0;` วนลูปตรวจทุกช่อง `totalSum += m[r][c];`
3. **คำนวณค่าเฉลี่ยทั้งหมด:** สมาชิกทั้งหมดมี $R \times C$ ตัว ดังนั้น:
   `double avg = (double) totalSum / (m.length * m[0].length);`
4. **คำนวณผลรวมรายแถว:**
   - ในลูปนอก (แต่ก่อนเริ่มลูปใน): ประกาศ `int rowSum = 0;`
   - ในลูปใน: บวกสะสม `rowSum += m[r][c];`
   - หลังจบลูปใน: พิมพ์ผลรวมของแถวนั้นออกมาทันที แล้วรอบถัดไปตัวแปร rowSum จะถูกรีเซ็ตเป็น 0 ใหม่อัตโนมัติ

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

```
เมทริกซ์ตัวอย่าง (2 แถว x 3 คอลัมน์):
[ 1, 2, 3 ] -> Row 0 Sum = 1 + 2 + 3 = 6
[ 4, 5, 6 ] -> Row 1 Sum = 4 + 5 + 6 = 15

ผลรวมทั้งหมด (Grand Total) = 6 + 15 = 21
จำนวนสมาชิกทั้งหมด = 2 x 3 = 6 ตัว
ค่าเฉลี่ยทั้งหมด (Average) = 21.0 / 6 = 3.50
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **คำนวณผลรวมรายคอลัมน์ (Column Sums):** สลับลูปเอา col ขึ้นก่อน แล้วสะสม `colSum += m[r][c]`
- **ผลรวมเส้นทแยงมุมหลัก (Trace of Matrix):** ในเมทริกซ์จัตุรัส วนลูปเดียว `for (int i=0; i`
- **หาแถวที่มีผลรวมมากที่สุด (ArgMax Row):** เก็บ `maxRowIndex`

⚠️ จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)

ประกาศตัวแปรสะสมผลรวมแถวผิดที่: ประกาศ `int rowSum = 0;` ไว้นอกลูปนอก ส่งผลให้ผลรวมของแถวที่ 0 ถูกนำไปบวกสะสมรวมต่อในแถวที่ 1 ทำให้ผลรวมแถวหลังๆ มีค่าเกินจริงเสมอ ตัวแปรสะสมรายแถวต้องประกาศ **อยู่ภายในลูปนอกเหนือลูปใน** เสมอ

ข้อที่ 35: Ex 4: การหาค่ามากที่สุดของแต่ละแถว (Maximum Element per Row)

Row-wise Search & Optimization | Lab11B_2DArrays/MatrixRowMax.java

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

สร้างอัลกอริทึมค้นหาค่าสูงสุดแยกตามแต่ละแถวของเมทริกซ์ โดยรีเซ็ตค่าเริ่มต้นสำหรับแต่ละแถวอย่างถูกต้อง

ซอร์สโค้ดฉบับสมบูรณ์ (MatrixRowMax.java):

```
// SE101 - Lab 11B: Exercise 4 (MatrixRowMax.java)
import java.util.Scanner;

public class MatrixRowMax {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter rows: ");
        int rows = scanner.nextInt();
        System.out.print("Enter cols: ");
        int cols = scanner.nextInt();

        int[][] matrix = readMatrix(rows, cols);

        System.out.println("\nMatrix:");
        printMatrix(matrix);

        System.out.println();
        findRowMax(matrix);

        scanner.close();
    }

    public static int[][] readMatrix(int rows, int cols) {
        Scanner scanner = new Scanner(System.in);
        int[][] m = new int[rows][cols];
        for (int row = 0; row < m.length; row++) {
            for (int col = 0; col < m[row].length; col++) {
                System.out.print("Enter element [" + row + "][" + col + "]: ");
                m[row][col] = scanner.nextInt();
            }
        }
        return m;
    }

    public static void printMatrix(int[][] m) {
        for (int row = 0; row < m.length; row++) {
            for (int col = 0; col < m[row].length; col++) {
                System.out.printf("%-5d", m[row][col]);
            }
            System.out.println();
        }
    }

    public static void findRowMax(int[][] m) {
        for (int row = 0; row < m.length; row++) {
            int maxInRow = m[row][0];
            for (int col = 1; col < m[row].length; col++) {
                if (m[row][col] > maxInRow) {
                    maxInRow = m[row][col];
                }
```

```
            }
            System.out.println("Max in row " + row + " = " + maxInRow);
        }
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. วนลูปตามแถว: `for (int r = 0; r < m.length; r++)`
2. **กำหนดค่าเริ่มต้น Max ของแถวนั้น:**
   `int rowMax = m[r][0];` (ต้องใช้สมาชิกคอลัมน์แรกของแถวนั้นๆ เสมอ)
3. วนลูปตรวจสอบคอลัมน์ที่เหลือ (เริ่มที่ `c = 1`):
   ```
   for (int c = 1; c < m[r].length; c++) {
       if (m[r][c] > rowMax) rowMax = m[r][c];
   }
   ```
4. แสดงผลลัพธ์ค่าสูงสุดของแถว `r`
5. วนไปทำแถวถัดไปจนครบทุกแถว

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

```
เมทริกซ์ขนาด 3x3:
แถว 0: [ 12,  45,  33 ] -> ค่าสูงสุดคือ 45
แถว 1: [  8,  99,  14 ] -> ค่าสูงสุดคือ 99
แถว 2: [ 70,  55,  62 ] -> ค่าสูงสุดคือ 70

หน้าจอ Output:
Max in row 0 = 45
Max in row 1 = 99
Max in row 2 = 70
```

🔄 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **หาค่าน้อยที่สุดของแต่ละแถว (Row Min):** เปลี่ยนเป็น `if (m[r][c] < rowMin) rowMin = m[r][c];`
- **หาค่ามากที่สุดของทั้งเมทริกซ์ (Global Max):** ประกาศตัวแปร `globalMax = m[0][0];` พร้อมเก็บพิกัด `maxR = 0, maxC = 0;`
- **คืนค่าเป็น 1D Array ของค่าสูงสุด:** `public static int[] getRowMaxes(int[][] m)`

⚠️ จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)

ตั้งค่าเริ่มต้น `int rowMax = 0;` หากในแถวนั้นมีเฉพาะตัวเลขติดลบทั้งหมด เช่น `[-10, -50, -25]` โปรแกรมจะตอบว่าค่าสูงสุดคือ `0` ซึ่งไม่มีอยู่จริงในแถวนั้น! ต้องตั้ง `rowMax = m[r][0]` เสมอ

ข้อที่ 36: Ex 5: การค้นหาข้อมูลในเมทริกซ์ 2 มิติ แบบหยุดลูปซ้อนทันที (2D Early Exit Search)

2D Search & Clean Early Exit Lab11B_2DArrays/MatrixSearch.java

🎯 จุดประสงค์และเป้าหมายของโปรแกรม

สร้างอัลกอริทึมค้นหาค่าเป้าหมาย (Target) ในเมทริกซ์ 2 มิติ และเทคนิคการหยุดการทำงานของลูปซ้อน 2 ชั้นทันทีที่พบเป้าหมายโดยไม่ใช้คำสั่ง break ซ้อนที่ซับซ้อน

ซอร์สโค้ดฉบับสมบูรณ์ (MatrixSearch.java):

```
// SE101 - Lab 11B: Exercise 5 (MatrixSearch.java)
import java.util.Scanner;

public class MatrixSearch {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter rows: ");
        int rows = scanner.nextInt();
        System.out.print("Enter cols: ");
        int cols = scanner.nextInt();

        int[][] matrix = readMatrix(rows, cols);

        System.out.println("\nMatrix:");
        printMatrix(matrix);

        System.out.print("\nEnter target: ");
        int target = scanner.nextInt();

        System.out.println("\n--- Search with Early Exit ---");
        search(matrix, target);

        System.out.println("\n--- Extension: Search All Occurrences ---");
        searchAll(matrix, target);

        scanner.close();
    }

    public static int[][] readMatrix(int rows, int cols) {
        Scanner scanner = new Scanner(System.in);
        int[][] m = new int[rows][cols];
        for (int row = 0; row < m.length; row++) {
            for (int col = 0; col < m[row].length; col++) {
                System.out.print("Enter element [" + row + "][" + col + "]: ");
                m[row][col] = scanner.nextInt();
            }
        }
        return m;
    }

    public static void printMatrix(int[][] m) {
        for (int row = 0; row < m.length; row++) {
            for (int col = 0; col < m[row].length; col++) {
                System.out.printf("%-5d", m[row][col]);
            }
            System.out.println();
        }
```

```
    }

    // Standard search with early exit
    public static void search(int[][] m, int target) {
        boolean found = false;
        for (int row = 0; row < m.length && !found; row++) {
            for (int col = 0; col < m[row].length && !found; col++) {
                if (m[row][col] == target) {
                    System.out.println("Found " + target + " at [" + row + "][" + col + "]");
                    found = true;
                }
            }
        }
        if (!found) {
            System.out.println(target + " not found.");
        }
    }

    // Extension: find and print ALL occurrences
    public static void searchAll(int[][] m, int target) {
        int occurrences = 0;
        for (int row = 0; row < m.length; row++) {
            for (int col = 0; col < m[row].length; col++) {
                if (m[row][col] == target) {
                    System.out.println("Found " + target + " at [" + row + "][" + col + "]");
                    occurrences++;
                }
            }
        }
        if (occurrences == 0) {
            System.out.println(target + " not found.");
        } else {
            System.out.println("Total occurrences found: " + occurrences);
        }
    }
}
```

⚙️ หลักการทำงานอย่างละเอียด (Step-by-Step Logic)

1. รับค่าเป้าหมายที่ต้องการค้นหา `int target = sc.nextInt();`
2. สร้างตัวแปรสถานะ: `boolean found = false;`
3. **เทคนิค Early Exit ใน Nested Loops:** ใส่เงื่อนไข `&& !found` ไว้ในหัวลูปทั้งสองชั้น:

```
for (int r = 0; r < m.length && !found; r++) {
    for (int c = 0; c < m[r].length && !found; c++) {
        if (m[r][c] == target) {
            System.out.printf("Found %d at row %d, column %d%n", target, r, c);
            found = true; // หยุดทั้งลูปในและลูปนอกทันที!
        }
    }
}
```

4. หลังจบลูป: ตรวจสอบ `if (!found) System.out.println(target + " not found in matrix.");`

📊 ตัวอย่างการทำงานและการ Trace ตัวแปร (Trace Table & Walkthrough)

```
ค้นหา target = 42 ในเมทริกซ์:
[  5, 12, 19 ]
[ 28, 42, 55 ]
[ 63, 71, 89 ]
```

```
การทำงาน:
- แถว 0: ตรวจ 5, 12, 19 -> ไม่ตรง
- แถว 1: ตรวจ 28 -> ไม่ตรง
          ตรวจ 42 -> พบแล้ว! พิมพ์พิกัด [1][1] และตั้ง found = true
- ลูปในรอบถัดไป: ตรวจเงื่อนไข c < 3 && !found -> !found เป็น false -> หลุดลูปในทันที
- ลูปนอกรอบถัดไป: ตรวจเงื่อนไข r < 3 && !found -> !found เป็น false -> หลุดลูปนอกทันที!

ผลลัพธ์: พบข้อมูลทันทีโดยไม่ต้องตรวจเลข 55, 63, 71, 89 ที่เหลือ
```

🔁 แบบต่างๆ และรูปแบบที่อาจเจอในข้อสอบ (Variations & Exam Tweaks)

- **ใช้ Labeled Break:** `searchLoop: for (...) { for (...) { if (...) { found = true; break searchLoop; } } }`
- **แยกเป็น Method ค้นหาและ return พิกัด:** `public static int[] find(int[][] m, int target) { for(...) for(...) if(m[r][c]==target) return new int[]{r, c}; return null; }` (วิธีนี้สะอาดที่สุด เพราะ `return` สั่งออกจากทั้งสองลูปในทันที)
- **นับจำนวนความถี่ที่พบทั้งหมด:** ตัด early exit ออก แล้วใช้ตัวแปรนับแทน

⚠️ จุดที่ต้องระวัง & กับดักข้อสอบ (Common Mistakes & Traps)

ใช้คำสั่ง break ตัวเดียวโดดๆ: คำสั่ง `break;` ใน Java จะกระโดดออกจากลูปที่อยู่ใกล้ที่สุดเพียงชั้นเดียว (ลูปคอลัมน์) แต่ *ลูปแถวยังคงทำงานต่อไป* ทำให้โปรแกรมยังคงวนค้นหาแถวถัดไปเรื่อยๆ จนจบเมทริกซ์ หากต้องการให้ออกทั้งหมดต้องใช้เทคนิค `&& !found`, Labeled Break หรือแยกเป็น Method แล้วใช้ `return`

บทที่ 7: วิเคราะห์และเฉลยข้อสอบตัวอย่าง Final Lab Exam (Model Solutions)

ข้อสอบ Final Lab Exam (จาก Example-FLabExam.pdf) ประกอบด้วยโจทย์มาตรฐาน 3 ข้อ ครอบคลุม Loops, Methods และ Arrays อย่างครบถ้วน

ข้อสอบที่ 1: Loops & Statistics — Temperature Converter with Statistics

โจทย์ข้อสอบ: โปรแกรมแปลงอุณหภูมิเซลเซียสเป็นฟาเรนไฮต์ พร้อมระบุหมวดหมู่ความร้อน ทำงานซ้ำจนกว่าผู้ใช้จะป้อนค่า -999 ตอนจบให้แสดงจำนวนครั้งที่แปลงและค่าฟาเรนไฮต์ที่สูงที่สุด

> 📐 **สูตรและเงื่อนไข**
>
> `F = (9.0 / 5.0) * C + 32`
>
> หมวดหมู่: `F < 50 (Cold)`, `50 <= F <= 86 (Comfortable)`, `F > 86 (Hot)`

Model Solution ที่ถูกต้องและกระชับที่สุดในห้องสอบ:

```
import java.util.Scanner;

public class TempExam {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int count = 0;
        double highestF = Double.NEGATIVE_INFINITY; // เริ่มต้นที่ค่าน้อยสุด

        while (true) {
            System.out.print("Enter Celsius (-999 to quit): ");
            double c = sc.nextDouble();
            if (c == -999) { // 1. ตรวจสอบ Sentinel ก่อนประมวลผลเสมอ!
                break;
            }

            double f = (9.0 / 5) * c + 32;
            String category;
            if (f < 50) {
                category = "Cold";
            } else if (f <= 86) {
                category = "Comfortable";
            } else {
                category = "Hot";
            }

            System.out.printf("%.1f °C = %.2f °F → %s%n", c, f, category);
            count++;
            if (f > highestF) {
                highestF = f;
            }
        }

        System.out.println("----------------------------");
        System.out.println("Total conversions : " + count);
        if (count > 0) {
            System.out.printf("Highest Fahrenheit: %.2f °F%n", highestF);
        } else {
            System.out.println("No data entered.");
        }
        sc.close();
    }
}
```

> 🔍 **วิเคราะห์เจาะลึกตรรกะในห้องสอบ**
>
> - ใช้ `while (true)` เพื่อรับค่าวนซ้ำไม่จำกัดรอบ
> - เช็ค `c == -999` ทันทีหลังรับค่า ถ้าตรงเงื่อนไขสั่ง `break;` ทันทีเพื่อไม่ให้ -999 ถูกนับและนำไปหาค่า Max

- ใช้ `(9.0 / 5)` เพื่อป้องกัน Integer Division
- ใช้ตัวแปร `highestF = Double.NEGATIVE_INFINITY` หรือรับค่าตัวแรกมาตั้งต้น เพื่อรองรับอุณหภูมิที่ติดลบได้
- จัดรูปแบบทศนิยมด้วย `%.1f` และ `%.2f` ตามที่โจทย์กำหนด

🔄 รูปแบบพลิกแพลงที่เป็นไปได้ในห้องสอบ

- **เปลี่ยน Sentinel:** โจทย์อาจเปลี่ยนจาก `-999` เป็น `0` หรือค่าลบใดๆ ก็ตาม
- **เพิ่มตัวสถิติ:** โจทย์อาจสั่งให้หาอุณหภูมิต่ำสุด (Lowest) หรืออุณหภูมิเฉลี่ย (Average Fahrenheit)
- **ใช้ do-while:** ปรับโครงสร้างลูปให้ทำก่อนแล้วค่อยเช็คเงื่อนไข

ข้อสอบที่ 2: Methods & Input Validation — Shipping Cost Calculator

โจทย์ข้อสอบ: สร้าง Method shippingCost(double weight, String type) เพื่อคำนวณค่าจัดส่งพัสดุตามน้ำหนักและปลายทาง (D: ในประเทศ 30 บ./กก. ขั้นต่ำ 50 บ., I: ต่างประเทศ 80 บ./กก. ขั้นต่ำ 150 บ.) ทำซ้ำจนกว่า weight = 0

> 📐 **สูตรและเงื่อนไข**
>
> D: weight * 30 (min 50)
> I: weight * 80 (min 150)

Model Solution ที่ถูกต้องและกระชับที่สุดในห้องสอบ:

```
import java.util.Scanner;

public class ShippingExam {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        while (true) {
            System.out.print("Enter weight (kg), 0 to quit: ");
            double weight = sc.nextDouble();
            if (weight == 0) {
                System.out.println("Goodbye!");
                break;
            }
            if (weight < 0) {
                System.out.println("Invalid weight.");
                continue; // ข้ามรอบนี้ไปรับน้ำหนักใหม่
            }

            System.out.print("Destination (D/I): ");
            String type = sc.next().toUpperCase(); // ปรับเป็นพิมพ์ใหญ่

            double cost = shippingCost(weight, type);
            if (cost == -1) {
                System.out.println("Invalid destination type.");
                continue;
            }

            // ตรวจสอบว่าใช้อัตราขั้นต่ำหรือไม่เพื่อพิมพ์ข้อความ
            double raw = type.equals("D") ? weight * 30 : weight * 80;
            double min = type.equals("D") ? 50 : 150;
            if (raw < min) {
                System.out.printf("Shipping cost = %.2f baht (minimum applied)%n", cost);
            } else {
                System.out.printf("Shipping cost = %.2f baht%n", cost);
            }
        }
        sc.close();
    }

    public static double shippingCost(double weight, String type) {
        if (type.equals("D")) {
            double cost = weight * 30;
            return (cost < 50) ? 50 : cost;
        } else if (type.equals("I")) {
            double cost = weight * 80;
            return (cost < 150) ? 150 : cost;
        }
        return -1; // ประเภทปลายทางไม่ถูกต้อง
```

```
    }
}
```

🔍 วิเคราะห์เจาะลึกตรรกะในห้องสอบ

- Method shippingCost รับ parameter 2 ตัวและ return ค่า double
- เปรียบเทียบ String ด้วย .equals("D") ห้ามใช้ ==
- ใช้ weight < 0 เช็ค validation แล้วใช้ continue; ข้ามรอบการทำงาน
- ใช้ Ternary Operator คำนวณค่าธรรมเนียมขั้นต่ำอย่างกระชับ
- ลูปใน main() วนซ้ำจนกว่า weight จะเท่ากับ 0

🔄 รูปแบบพลิกแพลงที่เป็นไปได้ในห้องสอบ

- **เพิ่มประเภทขนส่ง:** เช่น E: Express กก.ละ 120 บ. ขั้นต่ำ 200 บ.
- **ให้ส่วนลด:** ถ้าน้ำหนักเกิน 10 กก. ลดค่าส่ง 10%
- **ใช้ char แทน String:** char type = sc.next().toUpperCase().charAt(0);

ข้อสอบที่ 3: 1D Array & Comprehensive Report — Weekly Rainfall Report

โจทย์ข้อสอบ: รับปริมาณน้ำฝน 7 วันเก็บใน Array ขนาด 7 แล้วแสดง: 1. ปริมาณฝนแต่ละวัน 2. ผลรวม 3. ค่าเฉลี่ย 4. จำนวนวันที่ฝนตกเกินค่าเฉลี่ย (พร้อมระบุชื่อวัน) 5. วันที่ฝนตกหนักที่สุด (Day 1 = index 0)

> 📐 **สูตรและเงื่อนไข**
>
> Avg = Total / 7.0
> Above Avg: rain[i] > avg
> Max Day: index of max + 1

Model Solution ที่ถูกต้องและกระชับที่สุดในห้องสอบ:

```
import java.util.Scanner;

public class RainfallExam {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[] rain = new int[7];
        int total = 0;

        for (int i = 0; i < rain.length; i++) {
            System.out.print("Enter rainfall for day " + (i + 1) + ": ");
            rain[i] = sc.nextInt();
            total += rain[i];
        }

        System.out.println("\n--- Weekly Rainfall Report ---");
        for (int i = 0; i < rain.length; i++) {
            System.out.println("Day " + (i + 1) + ": " + rain[i] + " mm");
        }

        double avg = (double) total / rain.length;
        System.out.println("-----------------------------");
        System.out.println("Total rainfall : " + total + " mm");
        System.out.printf("Average        : %.2f mm%n", avg);

        // หาจำนวนวันที่ฝนตกมากกว่าค่าเฉลี่ย
        int aboveCount = 0;
        String daysAbove = "";
        for (int i = 0; i < rain.length; i++) {
            if (rain[i] > avg) {
                aboveCount++;
                if (!daysAbove.isEmpty()) {
                    daysAbove += ", ";
                }
                daysAbove += "Day " + (i + 1);
            }
        }
        System.out.println("Days above average: " + aboveCount + " (" + daysAbove + ")");

        // หาตำแหน่งวันที่ฝนตกมากที่สุด (ArgMax)
        int maxIdx = 0;
        for (int i = 1; i < rain.length; i++) {
            if (rain[i] > rain[maxIdx]) {
                maxIdx = i;
            }
        }
        System.out.println("Highest rainfall : Day " + (maxIdx + 1) + " (" + rain[maxIdx] + " mm)");
        sc.close();
```

```
    }
}
```

🔍 วิเคราะห์เจาะลึกตรรกะในห้องสอบ

- สร้าง Array `int[] rain = new int[7];`
- คำนวณผลรวมสะสม `total += rain[i]` ขณะที่รับ input ช่วยลดลูป
- คำนวณค่าเฉลี่ยโดยแปลง `(double) total / rain.length`
- ลูปครั้งที่สองกรองเงื่อนไข `rain[i] > avg` และต่อ String ชื่อวันด้วยคอมมา
- เทคนิค **ArgMax**: เก็บ index ของค่าสูงสุด `maxIdx` แทนการเก็บเฉพาะค่าตัวเลข เพื่อให้สามารถอ้างอิงกลับมาเป็นหมายเลขวันได้ด้วย `maxIdx + 1`

🔄 รูปแบบพลิกแพลงที่เป็นไปได้ในห้องสอบ

- **หาทั้ง Min และ Max:** วันที่ฝนตกน้อยที่สุด และวันที่ฝนตกมากที่สุด
- **รับขนาดจากผู้ใช้:** เปลี่ยนจาก 7 วัน เป็นให้ผู้ใช้ป้อน `int n = sc.nextInt();`
- **จัดกลุ่มเป็น Method:** แยก `calculateTotal`, `findMaxDay` ออกเป็นฟังก์ชันย่อย

บทที่ 8: Top 10 ข้อผิดพลาดร้ายแรงในห้องสอบ (Fatal Exam Pitfalls)

ข้อผิดพลาด 10 ประการที่ทำให้นักศึกษาถูกหักคะแนนมากที่สุดจากการสอบคอมโปร Java

🔥 อันดับที่ 1: Integer Division (การหารจำนวนเต็มตัดเศษ)

เมื่อจำนวนเต็มหารกับจำนวนเต็ม Java จะตัดเศษทิ้งก่อนนำไปเก็บในตัวแปร double เสมอ ต้อง cast ตัวใดตัวหนึ่งเป็น double ก่อน

❌ **เขียนแบบนี้ (ผิด / โดนหักคะแนน):**

```
int sum = 25, n = 4;
double avg = sum / n; // ได้ 6.0 ไม่ใช่ 6.25!
```

✅ **เขียนแบบนี้ (ถูกต้อง / เต็ม 100%):**

```
double avg = (double) sum / n; // ได้ 6.25 ถูกต้อง
```

🔥 อันดับที่ 2: ArrayIndexOutOfBoundsException (Index เกินขนาด Array)

Array ขนาด 5 จะมี Index ตั้งแต่ 0 ถึง 4 เท่านั้น การใช้ <= arr.length จะทำให้โปรแกรมหยุดทำงานกะทันหัน

❌ **เขียนแบบนี้ (ผิด / โดนหักคะแนน):**

```
int[] arr = new int[5];
for (int i = 0; i <= arr.length; i++) { ... } // พังที่รอบสุดท้าย!
```

✅ **เขียนแบบนี้ (ถูกต้อง / เต็ม 100%):**

```
for (int i = 0; i < arr.length; i++) { ... } // ถูกต้องเสมอ
```

🔥 อันดับที่ 3: Sentinel Value เช็คช้ากว่าการประมวลผล

Sentinel Value มีไว้เพื่อบอกให้หยุดโปรแกรม ไม่ใช่ข้อมูลจริง ต้องตรวจสอบและ break ก่อนนำไปคำนวณ

❌ **เขียนแบบนี้ (ผิด / โดนหักคะแนน):**

```
while (true) {
    int val = sc.nextInt();
    sum += val; // พลาด! ค่า -999 ถูกนำไปบวกด้วย
    if (val == -999) break;
}
```

✅ **เขียนแบบนี้ (ถูกต้อง / เต็ม 100%):**

```
while (true) {
    int val = sc.nextInt();
    if (val == -999) break; // เช็คก่อนเสมอ!
    sum += val;
}
```

🔥 อันดับที่ 4: กำหนดค่าเริ่มต้น Min/Max เป็นศูนย์

การตั้งค่า min=0 หรือ max=0 จะล้มเหลวทันทีเมื่อข้อมูลทั้งหมดเป็นบวกหรือติดลบทั้งหมด

❌ **เขียนแบบนี้ (ผิด / โดนหักคะแนน):**

```
int min = 0; // ถ้าข้อมูลใน array มีแต่เลขบวก [10, 25, 40] min จะคงเป็น 0 ตลอดไป!
```

✅ **เขียนแบบนี้ (ถูกต้อง / เต็ม 100%):**

```
int min = arr[0]; // กำหนดค่าเริ่มต้นจากสมาชิกตัวแรกของ Array เสมอ
```

🔥 อันดับที่ 5: ใช้เครื่องหมาย ^ แทนการยกกำลัง

Java ไม่มีตัวดำเนินการยกกำลังในระดับ Syntax เครื่องหมาย ^ คือ XOR ต้องใช้ฟังก์ชัน Math.pow() เสมอ

❌ **เขียนแบบนี้ (ผิด / โดนหักคะแนน):**

```
int res = 2 ^ 3; // ใน Java จะคำนวณ Bitwise XOR ได้ผลลัพธ์คือ 1
```

✅ **เขียนแบบนี้ (ถูกต้อง / เต็ม 100%):**

```
double res = Math.pow(2, 3); // ได้ 8.0 ถูกต้อง
```

🔥 อันดับที่ 6: เปรียบเทียบ String ด้วยเครื่องหมาย ==

String เป็น Object ใน Java เครื่องหมาย == จะเทียบตำแหน่งหน่วยความจำ ต้องใช้ .equals() หรือ .equalsIgnoreCase() เท่านั้น

❌ **เขียนแบบนี้ (ผิด / โดนหักคะแนน):**

```
if (type == "D") { ... } // เปรียบเทียบ Memory Reference อาจให้ผลลัพธ์เป็น false
```

✅ **เขียนแบบนี้ (ถูกต้อง / เต็ม 100%):**

```
if (type.equals("D")) { ... } // เปรียบเทียบเนื้อหาตัวอักษร
```

🔥 อันดับที่ 7: ใส่ Semicolon (;) หลังหัว For หรือ While Loop

เครื่องหมาย ; ท้ายลูปถือเป็นคำสั่งว่าง (Empty statement) ทำให้บล็อกปีกกาด้านล่างไม่ถูกผูกกับลูป

❌ **เขียนแบบนี้ (ผิด / โดนหักคะแนน):**

```
for (int i = 0; i < 10; i++); // ลูปทำงานตัวเปล่า 10 รอบ
{
    System.out.println(i); // ผิดไวยากรณ์ หรือทำงานรอบเดียว
}
```

✅ **เขียนแบบนี้ (ถูกต้อง / เต็ม 100%):**

```
for (int i = 0; i < 10; i++) {
    System.out.println(i);
}
```

🔥 อันดับที่ 8: สับสนระหว่าง arr.length และ str.length()

จำง่ายๆ: Array เป็นโครงสร้างข้อมูลใช้ .length (ไม่มีวงเล็บ) ส่วน String เป็น Class ใช้ .length() (มีวงเล็บ)

❌ **เขียนแบบนี้ (ผิด / โดนหักคะแนน):**

```
int len = arr.length(); // ผิด! Array length เป็น Attribute ไม่มีวงเล็บ
int sLen = str.length; // ผิด! String length() เป็น Method มีวงเล็บ
```

✅ **เขียนแบบนี้ (ถูกต้อง / เต็ม 100%):**

```
int len = arr.length;
int sLen = str.length();
```

🔥 อันดับที่ 9: Reverse Array วนลูปจนสุดความยาว

การกลับด้าน Array แบบ In-Place ต้องวนลูปถึงแค่กึ่งกลาง (length / 2) มิฉะนั้นจะสลับกลับคืนสภาพเดิม

❌ **เขียนแบบนี้ (ผิด / โดนหักคะแนน):**

```
for (int i = 0; i < arr.length; i++) {
    // สลับ arr[i] กับ arr[arr.length - 1 - i]
} // สลับไปแล้วสลับกลับ ได้ array เดิม!
```

✅ **เขียนแบบนี้ (ถูกต้อง / เต็ม 100%):**

```
for (int i = 0; i < arr.length / 2; i++) {
    // สลับถึงแค่ครึ่งเดียวของ Array
}
```

🔥 อันดับที่ 10: สร้าง Scanner ซ้ำซ้อนภายใน Method ย่อย

การสร้าง Scanner(System.in) ซ้ำๆ สิ้นเปลืองหน่วยความจำ และอาจเกิดปัญหา buffer desync ได้ ควรส่ง instance เดียวกันผ่าน parameter

❌ **เขียนแบบนี้ (ผิด / โดนหักคะแนน):**

```
public static void readMatrix(int r, int c) {
    Scanner sc = new Scanner(System.in); // สร้างใหม่ทุกครั้งที่เรียก
}
```

✅ **เขียนแบบนี้ (ถูกต้อง / เต็ม 100%):**

```
public static void readMatrix(Scanner sc, int r, int c) {
    // รับ Scanner ตัวเดิมที่ส่งมาจาก main
}
```